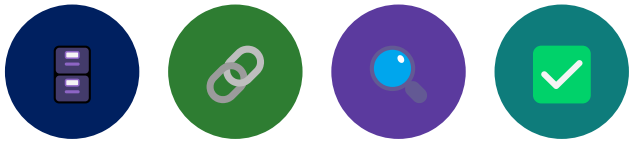
"Focus on your business logic, JPA takes care of the persistence."

MODULE 39

Spring Data JPA and PostgreSQL

Build powerful, data-driven applications using Spring Data JPA with PostgreSQL for robust, scalable, and maintainable enterprise solutions.




TEKsystems
Global Services
WORKFORCE
DEVELOPMENT


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT

MODULE 39 OVERVIEW

Build powerful, data-driven applications using Spring Data JPA with PostgreSQL for robust, scalable, and maintainable enterprise solutions.

This module moves the CRM platform's persistence from an in-memory repository to real PostgreSQL, using Spring Data JPA's entities, repositories, queries, and transactions.

DURATION & LAB

1 Hour Theory

JPA architecture, entity mapping, relationships, repositories, queries, pagination, JPQL, and transactions.

2.5 Hours Lab

lab39-crm: Flyway migrations, CustomerEntity/AccountEntity, repositories, paging, and optimistic locking against real PostgreSQL.

Scenario

Northstar CRM Customer Management Platform, wired to a shared PostgreSQL database with a Flyway-managed schema.

MODULE ARC

Understand JPA

Why ORM matters, JPA architecture, and the entity lifecycle.

Master Spring Data JPA

Entities, repositories, derived/custom queries, paging, sorting, and JPQL.

Build the Lab

Wire Spring Boot to real PostgreSQL with Flyway, entities, repositories, and optimistic locking.

KEY TAKEAWAY Total time: 3.5 hours (1 hour theory + 2.5 hours hands-on lab). By the end, you will build efficient, maintainable data access layers with Spring Data JPA and PostgreSQL.

WHY ORM MATTERS

Object-Relational Mapping bridges the gap between object-oriented code and relational databases -- domain objects instead of boilerplate SQL.

Without ORM	With ORM
Write repetitive SQL for CRUD operations.	Work with entities and repositories.
Handle result sets, mappings, and conversions by hand.	Automatic mapping and persistence.
Tight coupling with the database schema.	Loose coupling with the database.
Harder to maintain and test.	Easier maintenance and testing.
More code, more room for errors.	Less code, higher productivity.

KEY REASONS ORM MATTERS

1. Productivity

JPA reduces boilerplate and speeds up development with high-level abstractions.

2. Maintainability

Table changes need fewer code changes; business logic stays clean.

3. DB Independence

Easier to switch databases or evolve schema with minimal app impact.

4. Type Safety

Strongly typed entities and relationships instead of raw SQL strings.

5-6. Performance & Scale

Caching, lazy loading, batch operations, and standard scaling patterns.

KEY TAKEAWAY ORM doesn't replace SQL -- it abstracts it. You still benefit from the power of SQL while gaining the productivity of working with objects. Understand SQL and your database well.

Why ORM Matters

Without ORM
(Using Raw SQL / JDBC)

More Boilerplate Code
Writing SQL and handling ResultSets manually.

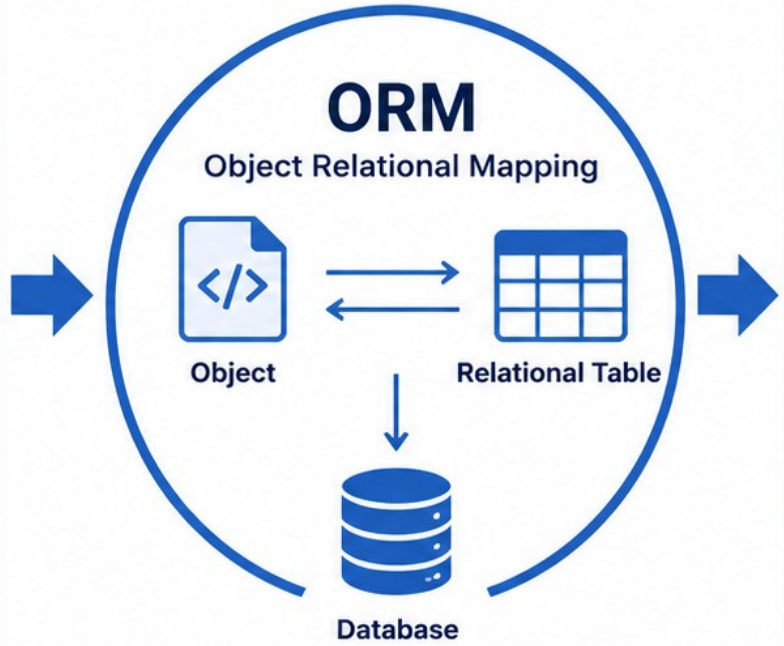
Tight Coupling with Database
SQL queries tied to a specific database schema.

Error-Prone
Prone to SQL errors and runtime exceptions.

Time Consuming
More development time for CRUD operations.

Hard to Maintain
Changes in schema require changes in many SQL queries.

Limited Productivity
Focus shifts from business logic to database interactions.



With ORM
(Using JPA / Hibernate)

Less Boilerplate Code
Simplifies CRUD operations with high-level abstractions.

Loose Coupling
Business logic is decoupled from database implementation.

Fewer Errors
Reduces SQL errors and improves application stability.

Faster Development
Increases productivity and speeds up delivery.

Easy Maintenance
Schema changes require minimal code updates.

Better Productivity
Focus on business logic, not on database code.

Bottom Line | ORM helps you write cleaner code, reduce development time, minimize errors, and build maintainable, database-independent applications.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to use Spring Data JPA with PostgreSQL to build efficient, maintainable, and enterprise-ready data access layers.

1. **Core Concepts —** explain the core concepts of JPA and how ORM works.
2. **Entity Design —** design and map JPA entities with proper relationships and constraints.
3. **CRUD via Repositories —** use Spring Data JPA repositories to perform CRUD operations efficiently.
4. **Queries —** write derived query methods and custom queries using JPQL.
5. **Large Datasets —** implement pagination and sorting to handle large datasets.
6. **Fetch Strategy —** explain lazy vs. eager loading and choose the right strategy.
7. **Connectivity —** configure and connect Spring Boot applications with PostgreSQL.
8. **Transactions —** manage transactions and ensure data consistency.
9. **Performance —** optimize JPA performance and avoid common ORM pitfalls.
10. **Best Practices —** apply enterprise best practices for robust and maintainable data access layers.

KEY TAKEAWAY Goal: build scalable, high-performance applications by leveraging the power of Spring Data JPA with PostgreSQL for enterprise-grade data persistence. Understand. Implement. Optimize. Excel.

WHAT IS JPA?

JPA (Java Persistence API) is a standard Jakarta EE specification defining how Java objects are mapped to relational database tables and how data is persisted.

IN SIMPLE TERMS

- JPA lets you work with Java objects instead of writing SQL for storing, retrieving, updating, and deleting data.
- It acts as a bridge between your object-oriented application and the relational database.

WHY USE JPA?

- Reduces boilerplate JDBC code.
- Improves developer productivity.
- Database independent (portable).
- Integrates seamlessly with Spring.
- Supports best practices and standards.

KEY CONCEPTS IN JPA

Entity — a Java class mapped to a database table.
Entity Manager — the primary interface to manage entities.
Persistence Context — a cache holding managed entities.
JPQL — a platform-independent query language.
Provider — the JPA implementation (e.g., Hibernate).
Transaction — a unit of work ensuring data consistency.

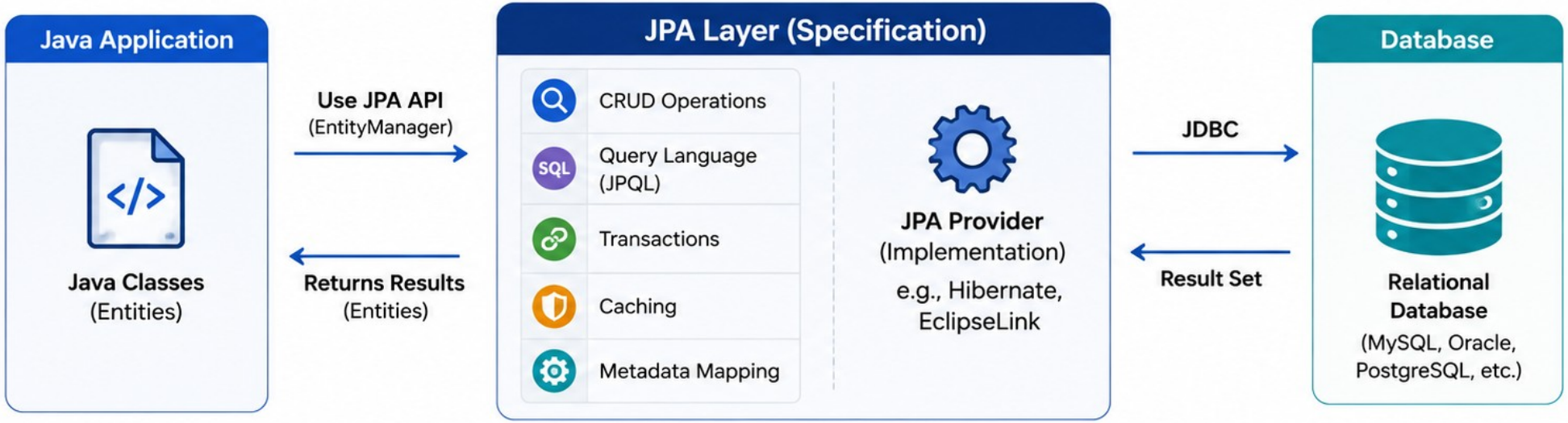
HOW JPA WORKS

1. **Java Application**
   Your domain objects.
2. **Entity Manager**
   Interface into JPA.
3. **Persistence Context**
   1st-level cache.
4. **JPA Provider**
   Hibernate generates SQL.
5. **PostgreSQL**
   Relational database.

KEY TAKEAWAY JPA provides a standard, efficient, and object-oriented way to persist data, making applications easier to develop, maintain, and scale.

What is JPA?

JPA (Java Persistence API) is a specification for managing relational data in Java applications. It provides an **object-relational mapping (ORM)** framework that simplifies data persistence, reduces JDBC boilerplate code, and promotes **database portability**.



Key Benefits

- Reduces JDBC boilerplate code
- Database independence
- Improved productivity
- Type-safe queries
- Better maintainability

Key Points

- Standard API (Jakarta Persistence)
- Works with any JPA Provider
- Maps Java objects to database tables
- Supports JPQL for object-oriented queries
- Simplifies transaction and lifecycle management

JPA ARCHITECTURE OVERVIEW

JPA defines a standard architecture for object-relational mapping. The JPA provider implements the specification and handles all persistence operations.

1. **Application Layer**
   Java app, no SQL/JDBC code.
2. **Entity Manager**
   Main entry point.
3. **Persistence Context**
   Tracks entity state.
4. **JPA Provider**
   Hibernate executes SQL.
5. **PostgreSQL**
   Tables, indexes, constraints.

KEY INTERACTIONS

- Application requests operations using EntityManager.
- EntityManager interacts with the Persistence Context.
- Changes are tracked in the context (managed entities).
- On transaction commit, the provider synchronizes changes to the database.
- Results from queries are mapped back to entities.
- Application works with objects, not database details.

BENEFITS

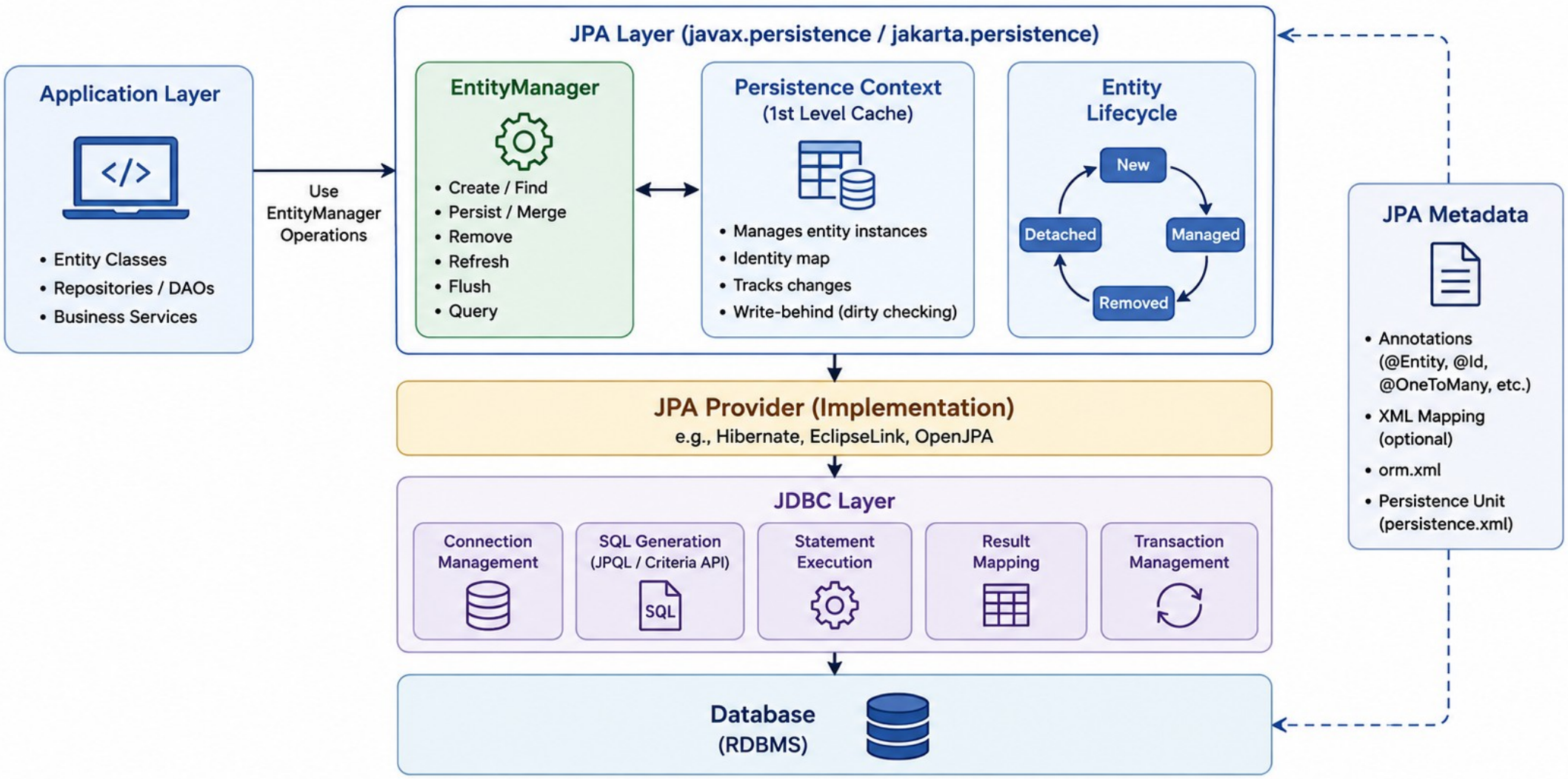
- Standard, portable API.
- Reduces boilerplate JDBC code.
- Higher developer productivity.
- Consistency, integrity, and seamless Spring integration.

METADATA / MAPPING INFORMATION

Annotations (or XML) define how Java classes map to database tables and columns -- this metadata drives every step of the flow above.

KEY TAKEAWAY JPA provides a clean separation between business logic and data access, making applications maintainable, testable, and database independent.

JPA Architecture Overview



ENTITY LIFECYCLE

An entity instance moves through Transient, Managed, Detached, and Removed states from creation until it is removed from the database.

1. Transient	2. Managed	3. Detached	4. Removed	5. Gone
Newly created with new; not associated with a Persistence Context; not in the database.	Associated with a Persistence Context; changes are tracked and synchronized automatically.	Was managed, no longer associated with a Persistence Context; changes untracked until re-managed.	Marked for removal; will be deleted on next flush or commit.	No longer exists in the database and is not associated with a Persistence Context.

STATE TRANSITIONS

Example: Transient -> Managed -> Removed

```
Customer c = new Customer();          // Transient
c.setName("Amina Khan");
entityManager.persist(c);             // Managed: tracked
entityManager.remove(c);              // Removed: pending delete
```

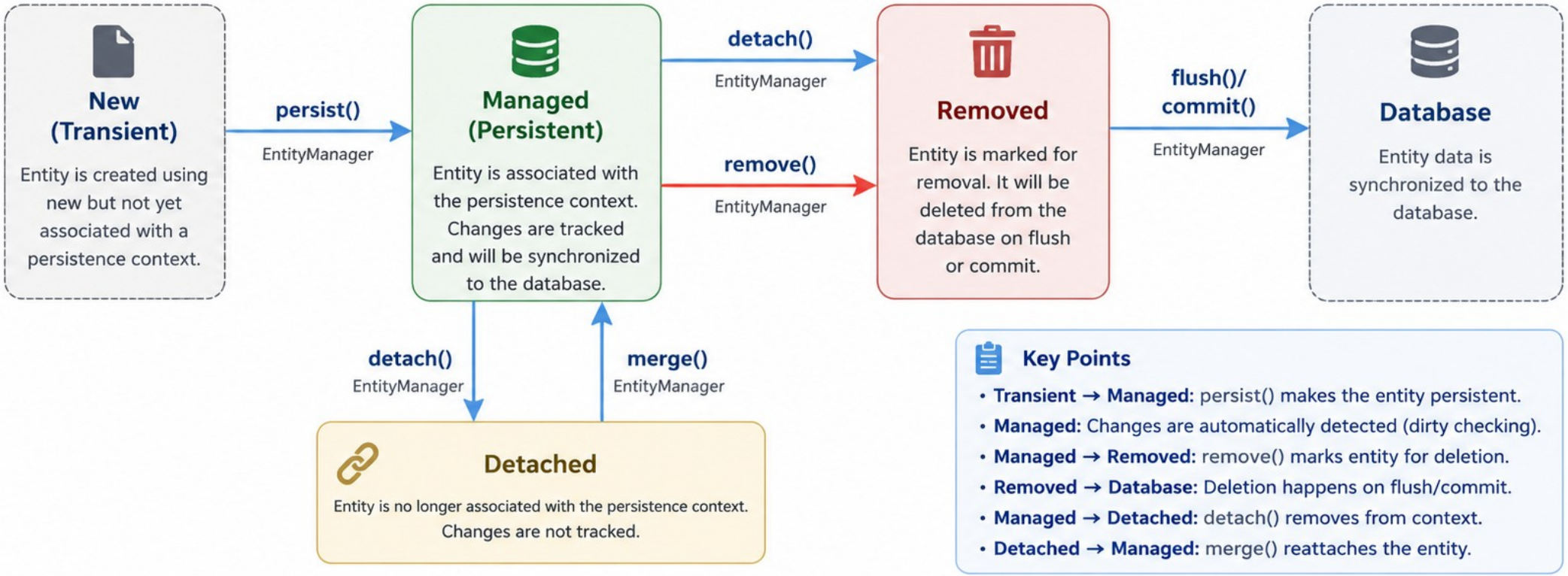
PRACTICAL TIPS

- Keep entities Managed only as long as needed.
- Use transactions to define persistence-context scope.
- Be careful with Detached entities in web apps.

KEY TAKEAWAY JPA automatically tracks entity state changes and synchronizes them with the database at the right time. Mastering the lifecycle is key to bug-free persistence logic.

Entity Lifecycle

The different states an entity goes through in the persistence context



CREATING JPA ENTITIES

JPA entities map your Java classes to PostgreSQL tables. They define the structure of your data and form the foundation of your Spring Data JPA application.

WHAT IS A JPA ENTITY?

- A plain Java class annotated with @Entity, mapped to a database table -- each instance represents one row.

KEY POINTS

- Must have a primary key.
- Requires a no-arg constructor.
- Mapped using annotations (or XML).
- Managed by the JPA provider (Hibernate).

CustomerEntity.java (PostgreSQL)

```
@Entity
@Table(name = "customer")
public class CustomerEntity {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  @Column(name = "customer_id")
  private Long id;
  @Column(name = "full_name", nullable = false)
  private String fullName;
}
```

IMPORTANT ANNOTATIONS

Annotation	Purpose
@Entity	Marks the class as a JPA entity.
@Table	Specifies the table name (PostgreSQL default lower_snake_case).
@Id	Defines the primary key field.
@GeneratedValue	Configures primary key generation (IDENTITY/SEQUENCE).
@Column	Maps a field to a column and customizes it.

BEST PRACTICES FOR POSTGRESQL

- Use meaningful, lowercase snake_case table and column names (PostgreSQL convention).
- Always define a primary key; prefer IDENTITY or SEQUENCE for generation.
- Use @Column to control nullability, length, and uniqueness.
- Keep entities focused on data, not business logic.
- Requires a no-arg constructor for JPA to instantiate the entity.

KEY TAKEAWAY Well-designed JPA entities are the key to a robust, maintainable, and high-performance PostgreSQL-backed application.

Creating JPA Entities

Map your Java classes to database tables using JPA annotations.

1. Define the Class

Create a plain Java class that represents the table.

```
public class User { ... }
```

2. Mark as Entity

Use @Entity to tell JPA this class is an entity.

```
@Entity
```

3. Define Primary Key

Use @Id to specify the primary key field.

```
@Id
```

4. Map Fields

Use annotations to map class fields to columns.

```
@Column(name = "...")
@Temporal, @Enumerated, etc.
```

5. (Optional) Table Name

Use @Table to specify table name and options.

```
@Table(name = "...")
```

Example: User Entity

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "username", nullable = false, length = 50, unique = true)
    private String username;

    @Column(name = "email", nullable = false, length = 100)
    private String email;

    @Column(name = "created_at", updatable = false)
    @Temporal(TemporalType.TIMESTAMP)
    private Date createdAt;
}
```

Resulting Table: users

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, Auto Increment	Primary Key
username	VARCHAR(50)	NOT NULL, UNIQUE	Username of the user
email	VARCHAR(100)	NOT NULL	Email address
created_at	TIMESTAMP	NOT NULL	Account creation time

Best Practices

- Use meaningful entity and field names.
- Always define a primary key.
- Specify constraints for data integrity.
- Keep entities focused on domain, not DB logic.

MAPPING DATABASE TABLES

JPA uses annotations to map Java classes and their fields to PostgreSQL database tables and columns.

1. CLASS TO TABLE MAPPING

- Use @Entity to mark a class as a JPA entity.
- Use @Table to specify the table name.
- If @Table is omitted, the class name is used.

2. FIELD TO COLUMN MAPPING

- Each field maps to a table column.
- Use @Column to customize name, length, nullability, uniqueness.
- If @Column is omitted, the field name is used.

3. PRIMARY KEY MAPPING

- Use @Id to mark the primary key field.
- Use @GeneratedValue to define how the key is generated.

Customer Entity Mapping (PostgreSQL)

```
import jakarta.persistence.*;

@Entity
@Table(name = "customers")
public class Customer {

  @Id
  @GeneratedValue(strategy =
      GenerationType.IDENTITY)
  @Column(name = "id")
  private Long id;

  @Column(name = "customer_name",
      nullable = false, length = 100)
  private String name;

  @Column(name = "email", unique = true,
      nullable = false, length = 100)
  private String email;

  @Column(name = "created_at",
      nullable = false, updatable = false)
  private LocalDateTime createdAt;
  // getters and setters
}
```

ENTITY <-> TABLE MAPPING

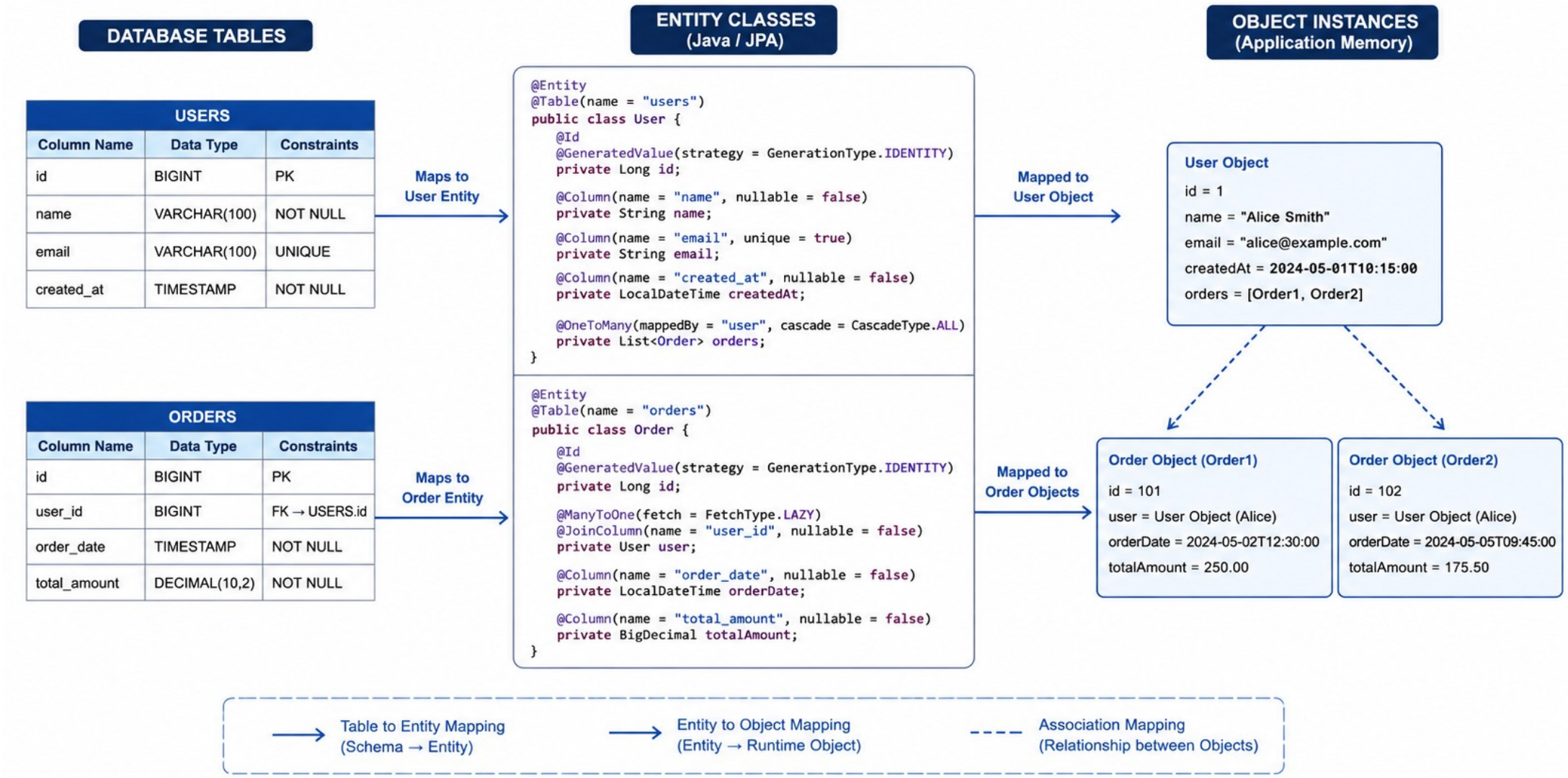
Field	Column
id	id (PK)
name	customer_name
email	email (unique)
createdAt	created_at

BEST PRACTICES

- Use meaningful table and column names.
- Keep entities focused on business data, not database details.
- Use appropriate PostgreSQL data types and constraints.
- Prefer auto-generation strategies for primary keys.

KEY TAKEAWAY Proper mapping ensures seamless interaction between Java objects and PostgreSQL database tables.

Mapping Database Tables



TRY IT YOURSELF — EXERCISE 1: ENTITY MAPPING

Reinforces: mapping Lab 37's customer/account columns to JPA fields and annotations you just saw on Mapping Database Tables, before primary key strategies.

STEPS (IN notes/lab39-jpa.md)

Step	What To Do
1 — Copy the Map	Copy the customer_id/full_name/status/created_at reference table into notes/lab39-jpa.md.
2 — Add Account	Add the account mapping: Long id, String customerId, plus a note for the coming @ManyToOne.
3 — Choose Naming	Decide and write down your snake_case column vs. camelCase field naming strategy.
4 — Anchor to Fixture	Mental-model one entity instance: customerId = "CUS-1001", fullName = "Amina Khan".

Reference mapping to fill in

```
@Id
private String customerId;              // customer_id
@Column(name = "full_name", nullable = false)
private String fullName;                // full_name
private String status;                  // status (or enum)
private Instant createdAt;              // created_at
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-entity-mapping.md (workspace: examples/module-39-exercises).

PRIMARY KEY STRATEGIES

A primary key uniquely identifies each record. JPA provides multiple strategies to generate or assign primary key values.

Strategy	Description	Example Annotation	PostgreSQL Behavior
IDENTITY	The database auto-increments the primary key (PostgreSQL waits for the DB to generate the value).	@GeneratedValue(strategy = GenerationType.IDENTITY)	Default and simplest on PostgreSQL 10+; DB generates value on INSERT.
SEQUENCE	Uses a database sequence object to generate IDs before insert.	@GeneratedValue(strategy = GenerationType.SEQUENCE)	Best performance for bulk inserts and distributed systems.
TABLE	Uses a dedicated table to store and generate primary key values.	@GeneratedValue(strategy = GenerationType.TABLE)	Database independent, but slower -- extra table lookup per ID.
AUTO	Lets JPA choose the best strategy based on the database dialect.	@GeneratedValue(strategy = GenerationType.AUTO)	Good default for portability across databases.

POSTGRESQL CONSIDERATIONS

- IDENTITY is the default and simplest choice on PostgreSQL 10+.
- SEQUENCE offers better performance for bulk inserts and distributed systems.
- Avoid TABLE strategy in high-concurrency environments due to contention.
- Use BIGINT (not plain INT) for scalability and larger data volume.

Lab 39: CustomerEntity primary key

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "customer_id")
private Long id;    // BIGINT GENERATED BY DEFAULT
                    // AS IDENTITY in Flyway V1
```

KEY TAKEAWAY Choose the right primary key strategy based on your database, scalability needs, and application requirements -- IDENTITY is the PostgreSQL default; SEQUENCE wins for high-throughput batch inserts.

Primary Key Strategies

Choosing the right primary key strategy impacts performance, scalability, and data integrity.

1. Surrogate Key (Auto-Increment)

Use a system-generated numeric value that has no business meaning.

Example:

Table: **Users**

id (PK)	name	email
1	Alice	alice@example.com
2	Bob	bob@example.com
3	Charlie	charlie@example.com

Use When:

- Simplicity and performance are priorities
- Data may change over time

2. UUID / GUID

Use a globally unique identifier generated by an algorithm.

Example:

Table: **Orders**

id (PK)	order_date	amount
9f3b2c4e-...	2024-05-01	120.00
a7d9c1e3-...	2024-05-02	250.00
b1e4f2a6-...	2024-05-03	75.00

Use When:

- Data is distributed across multiple systems
- Merging data from different sources

3. Natural Key

Use a real-world attribute (or set of attributes) that uniquely identifies the record.

Example:

Table: **Products**

sku (PK)	name	price
PRD-1001	Laptop	999.00
PRD-1002	Mouse	25.00
PRD-1003	Keyboard	45.00

Use When:

- The attribute is immutable and unique
- It has business meaning

4. Composite Key

Use a combination of two or more columns to uniquely identify a record.

Example:

Table: **OrderItems**

order_id (PK)	product_id (PK)	quantity
1001	501	2
1001	502	1
1002	501	3

Use When:

- No single column can uniquely identify the record
- Represents a many-to-many relationship

5. Snowflake / Distributed ID

Use a unique ID generated by a distributed algorithm (e.g., Snowflake IDs).

Example:

Table: **Events**

id (PK)	timestamp	source
64-bit ID	2024-05-01 10:00	Service A
64-bit ID	2024-05-01 10:01	Service B
64-bit ID	2024-05-01 10:02	Service C

Use When:

- Need high performance and scalability
- Operating in distributed environments

Key Takeaway:

There's no one-size-fits-all. Choose the strategy that best fits your system architecture, business requirements, and data lifecycle.

Performance | Data Integrity | Scalability | Maintainability

RELATIONSHIP MAPPING (1 OF 2): TYPES & ANNOTATIONS

JPA allows you to map relationships between entities using annotations. Understanding cardinality and ownership is essential to correct mapping.

Relationship	Description	Cardinality	Owning Side
One-to-One (@OneToOne)	A single entity is associated with only one entity of another type.	1 <-> 1	Either side; use @JoinColumn on owning side.
One-to-Many / Many-to-One (@OneToMany / @ManyToOne)	One entity relates to many entities. The many side owns the relationship.	1 <-> N	Many side (owning); defines the foreign key.
Many-to-Many (@ManyToMany)	Many entities relate to many entities of another type.	N <-> N	Either side; creates a join table automatically.
One-to-Many, Bidirectional	Both sides are aware of each other via mappedBy.	1 <-> N	Inverse side uses mappedBy; owning side unchanged.

RELATIONSHIP MAPPING ANNOTATIONS

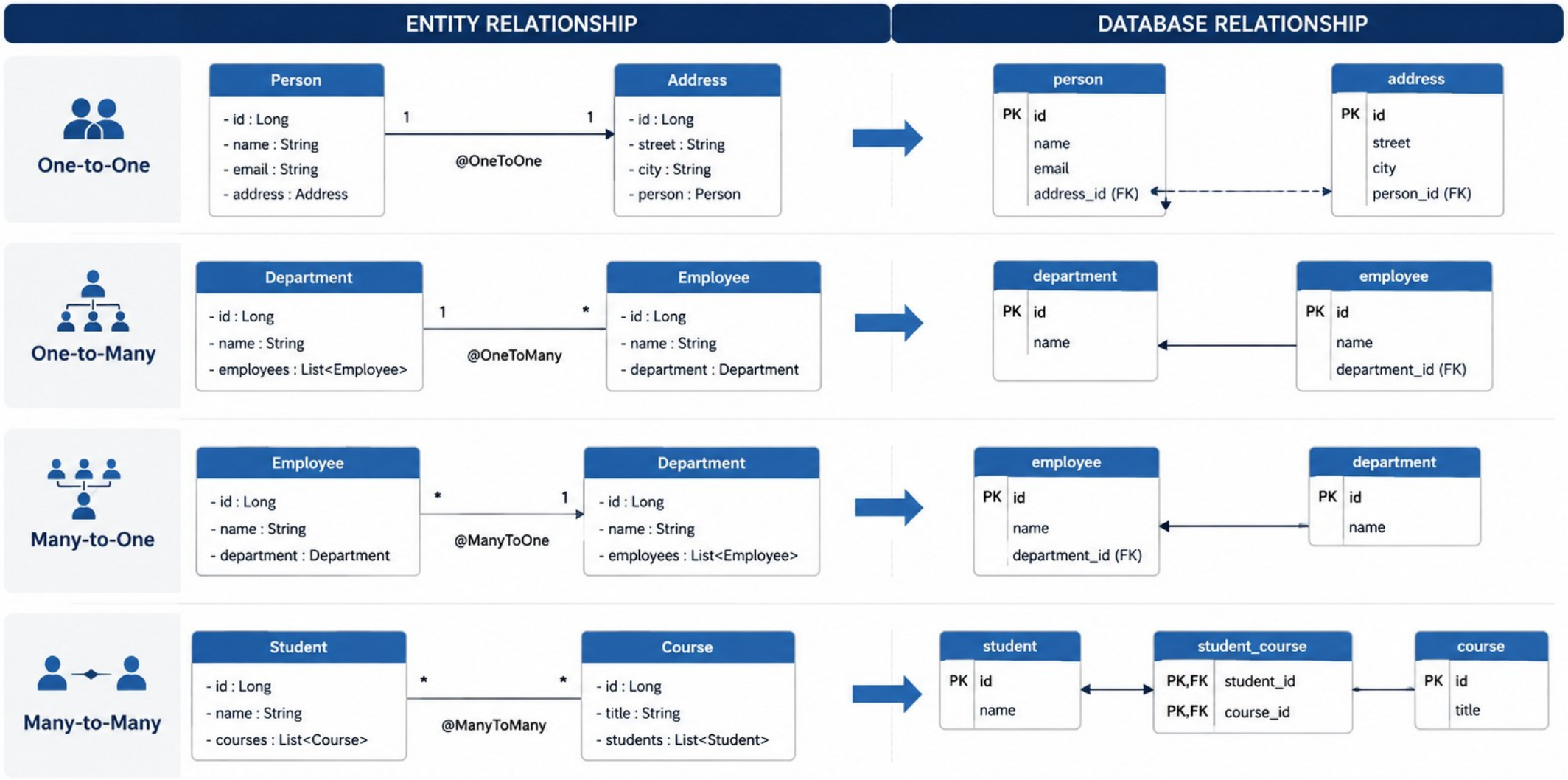
Annotation	Notes
@JoinColumn	Specifies the foreign key column; use with @ManyToOne/@OneToOne.
@JoinTable	Specifies the join table for @ManyToMany; customizes join columns.
mappedBy	Marks the inverse (non-owning) side of a bidirectional relationship.

KEY POINTS

- The owning side controls the foreign key.
- Use mappedBy on the inverse side.
- LAZY loading is recommended for collections.
- Choose fetch type and cascade options carefully.

KEY TAKEAWAY Model relationships that reflect real-world business rules; avoid bidirectional relationships unless necessary; the owning side always controls the foreign key.

Relationship Mapping



RELATIONSHIP MAPPING (2 OF 2): WORKED EXAMPLES

Lab 39's Customer-to-Account relationship is a One-to-Many / Many-to-One -- the pattern used throughout the CRM's real schema.

CustomerEntity.java (One side)

```
@Entity
@Table(name = "customer")
public class CustomerEntity {
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;
  private String fullName;

  @OneToMany(mappedBy = "customer", fetch = FetchType.LAZY)
  private Set<AccountEntity> accounts = new HashSet<>();
  // getters/setters -- accounts excluded from equals/toString
}
```

AccountEntity.java (Many / owning side)

```
@Entity
@Table(name = "account")
public class AccountEntity {
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @ManyToOne(fetch = FetchType.LAZY, optional = false)
  @JoinColumn(name = "customer_id", nullable = false)
  private CustomerEntity customer;

  @Column(precision = 19, scale = 2, nullable = false)
  private BigDecimal balance;
}
```

DATABASE REPRESENTATION

customer (id PK, full_name) 1 <-> N account (id PK, customer_id FK, balance). The foreign key customer_id lives on account -- the owning side.

COMMON PITFALLS

- Using EAGER fetch by default can hurt performance.
- Cascading REMOVE to child collections unintentionally.
- Infinite recursion in toString(), equals(), or JSON serialization of both sides.

KEY TAKEAWAY Well-designed relationships are the backbone of a robust domain model. The owning side (the Many side of @OneToMany/@ManyToOne) always holds the foreign key.

SPRING DATA REPOSITORY PATTERN

Spring Data JPA provides repository interfaces that handle data access operations for entities -- reducing boilerplate and improving productivity.

REPOSITORY HIERARCHY

Repository<T,ID> — marker interface.

CrudRepository<T,ID> — provides basic CRUD methods.

PagingAndSortingRepository — adds pagination and sorting.

JpaRepository<T,ID> — adds JPA-specific methods and query capabilities. Used throughout this course.

CustomerRepository.java

```
import org.springframework.data.jpa
    .repository.JpaRepository;

public interface CustomerRepository
    extends JpaRepository<CustomerEntity, Long> {

  Optional<CustomerEntity> findByPublicId(
      String publicId);

  boolean existsByNormalizedEmail(
      String normalizedEmail);

  Page<CustomerEntity> findByStatus(
      CustomerStatus status, Pageable pageable);
}
// No implementation class needed --
// Spring Data JPA generates it at runtime.
```

REPOSITORY IN ACTION

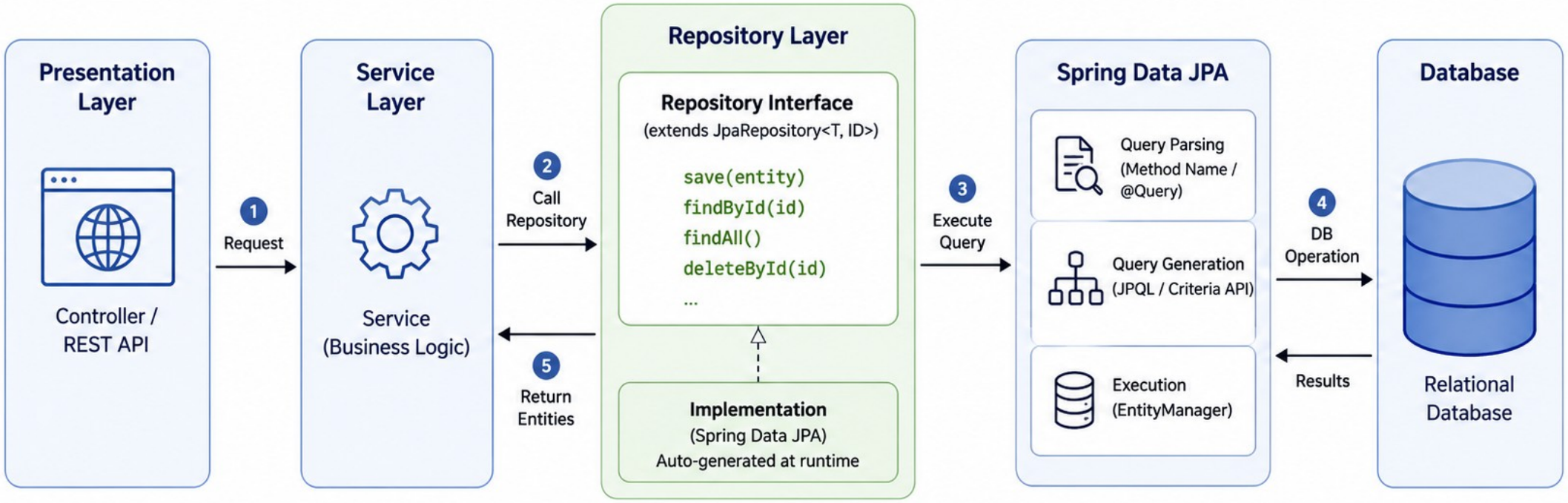
1. **Service Layer**
2. **CustomerRepository**
3. **Spring Data JPA**
4. **PostgreSQL**

BENEFITS

- No implementation classes to write.
- Out-of-the-box CRUD operations.
- Powerful query generation from method names.
- Supports pagination, sorting, and specifications.

KEY TAKEAWAY The Spring Data Repository Pattern simplifies data access and promotes clean, maintainable, and testable code -- extend JpaRepository, declare method signatures, done.

Spring Data Repository Pattern



How it works

1. Controller receives request and calls Service layer.
2. Service layer calls Repository interface methods.
3. Spring Data JPA parses the method name or @Query and generates query
4. Query is executed against the database via EntityManager.
5. Results are mapped to entities and returned to the Service layer.

Benefits

- Reduces boilerplate code
- Faster development
- Consistent data access layer
- Easy testing and maintenance

TRY IT YOURSELF — EXERCISE 2: REPOSITORY SKETCH

Reinforces: sketching real CustomerRepository/AccountRepository method signatures from Spring Data Repository Pattern, before CRUD Operations.

STEPS (IN notes/lab39-repos.md)

Step	What To Do
1 — CustomerRepository	List at least three methods: findById, findByStatus, findAll(Pageable).
2 — AccountRepository	Sketch findByCustomerId(String customerId) for Amina's and Ravi's accounts.
3 — Derived vs @Query	Note when a @Query would be clearer than a long derived method name.
4 — Layering Reminder	Write the rule: controllers talk to services; services use repositories.

Repository sketch to fill in

```
interface CustomerRepository extends JpaRepository<CustomerEntity, String> {
  Optional<CustomerEntity> findById(String customerId);
  Page<CustomerEntity> findByStatus(String status, Pageable pageable);
}
interface AccountRepository extends JpaRepository<AccountEntity, Long> {
  List<AccountEntity> findByCustomerId(String customerId);
}
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-repository-sketch.md (workspace: examples/module-39-exercises).

CRUD OPERATIONS WITH SPRING DATA JPA

Spring Data JPA makes Create, Read, Update, and Delete operations simple and consistent using repository interfaces.

Op	Method	Return Type	Description
Create	save(entity)	<S extends T> S	Inserts a new entity, or updates if the ID already exists.
Read	findById(id) / findAll() / count()	Optional<T> / List<T> / long	Finds by ID, retrieves all entities, or returns the total count.
Update	save(entity)	<S extends T> S	Updates an existing managed entity -- same method as Create.
Delete	deleteById(id) / delete(entity)	void / void	Deletes by ID or by the given entity instance.

CRUD ACRONYM

Create — insert new data.
Read — retrieve data.
Update — modify existing data.
Delete — remove data.

How CRUD flows

```
Controller/Service -> CustomerRepository ->
  Spring Data JPA generated impl ->
  JPA Provider (Hibernate) generates SQL ->
  PostgreSQL persists / retrieves
```

BEST PRACTICES

- Use meaningful method names for custom queries.
- Use Optional<T> to handle null-safe reads.
- Prefer deleteById() over loading then deleting.
- Keep transactions at the service layer (@Transactional).

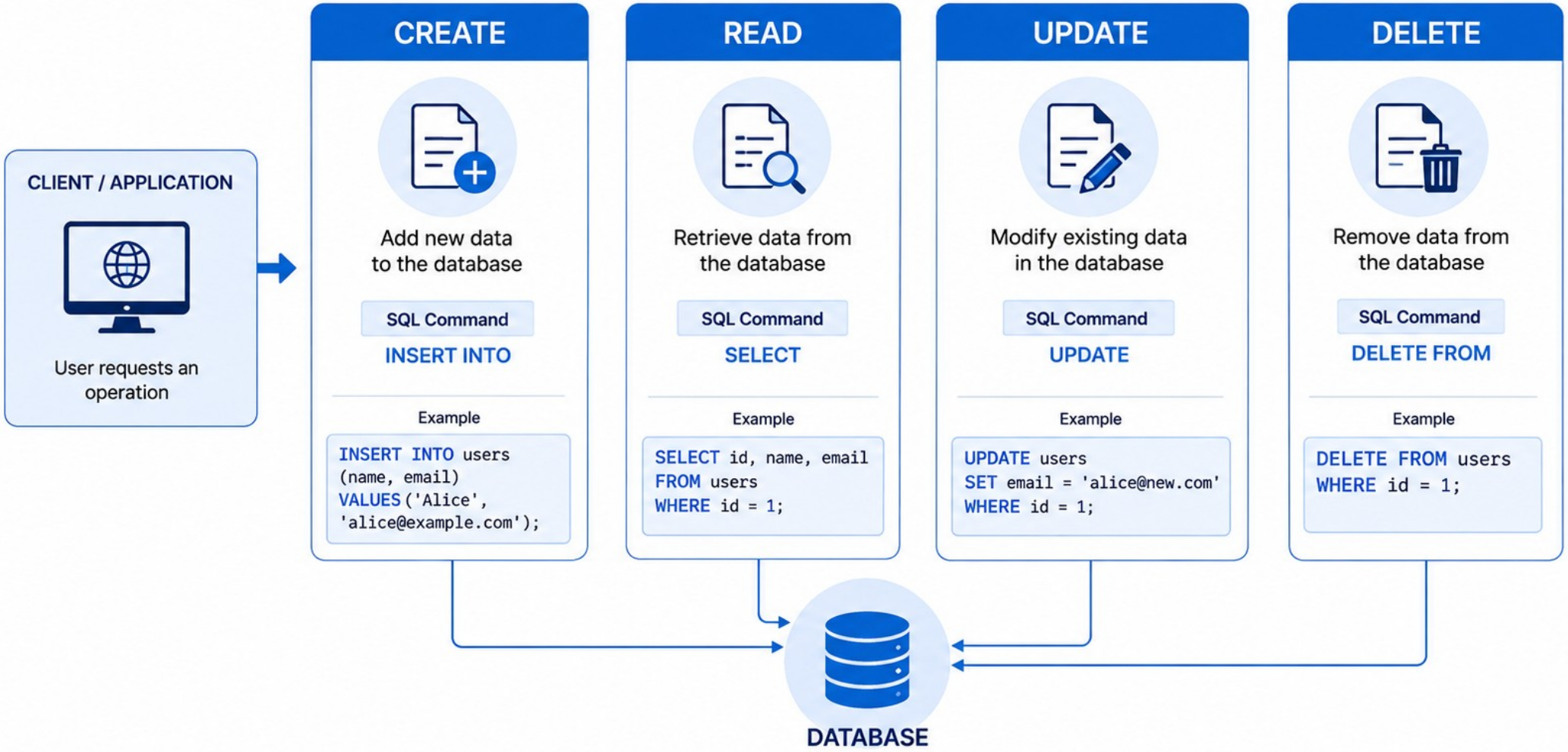
NOTE

JpaRepository<T, ID> provides many more methods beyond basic CRUD -- paging, sorting, and batch operations, plus custom methods via naming conventions or @Query.

KEY TAKEAWAY Spring Data JPA simplifies data access and lets you focus on building business logic instead of writing repetitive CRUD boilerplate.

CRUD Operations

Create, Read, Update, Delete – The 4 basic operations performed on data in a database.



TRY IT YOURSELF — EXERCISE 3: FILL JPA TODOS (STARTER)

Starter exercise -- fill in the blanks across an entity, its repository, and application.yml to lock in entity + repository + CRUD basics.

STEPS (IN notes/lab39-todos.md)

Step	What To Do
1 — Paste the Starter	Create notes/lab39-todos.md and paste the TODO pseudocode below.
2 — Fill the Blanks	Suggested: customerId, fullName, version, Pageable, a postgresql:// URL, validate (or none), true.
3 — Usage TODO	Write: service.load("CUS-1001") -> Optional<CustomerEntity> for Amina.
4 — Locking Note	Write: concurrent updates to Ravi's status bump @Version or fail with an optimistic lock exception.

Starter pseudocode -- fill every ____ (STARTER, not a complete solution)

```
@Entity @Table(name = "customer")
class CustomerEntity {
  @Id private String _____;
  @Column(name = "full_name", nullable = false) private String _____;
  @Version private long _____; // optimistic lock
}
interface CustomerRepository extends JpaRepository<CustomerEntity, String> {
  Page<CustomerEntity> findByStatus(String status, _____ pageable);
}
// application.yml: spring.jpa.hibernate.ddl-auto: _____   spring.flyway.enabled: _____
```

KEY TAKEAWAY TRY IT NOW (STARTER) — Complete Exercise 3 before continuing: exercises/exercise-03-fill-jpa-todos.md -- fill every ____ and // TODO; this is a starter, not a complete solution.

DERIVED QUERY METHODS

Spring Data JPA can automatically create queries based on method names -- no @Query annotation required for most common use cases.

CustomerRepository -- derived queries

```
// Find by a single field
Optional<CustomerEntity> findByPublicId(String id);
List<CustomerEntity> findByFullName(String name);

// Keywords: Containing, And, GreaterThan...
List<CustomerEntity> findByFullNameContaining(
    String name);                          // LIKE %name%
List<CustomerEntity> findByStatusAndCreatedAtAfter(
    CustomerStatus status, Instant since);

// Ordering and limiting
List<CustomerEntity> findTop10ByOrderByCreatedAtDesc();

// Existence and counting
boolean existsByNormalizedEmail(String email);
long countByStatus(CustomerStatus status);
```

COMMON KEYWORDS IN METHOD NAMES

Keyword	Translates To (SQL)
And / Or	WHERE a = ? AND/OR b = ?
Between	WHERE age BETWEEN ? AND ?
LessThan / GreaterThan	WHERE age < ? / age > ?
IsNull / IsNotNull	WHERE col IS [NOT] NULL
Containing / StartingWith	WHERE col LIKE %..% / ...%
OrderBy ... Desc/Asc	ORDER BY col DESC/ASC
Top / First	LIMIT n

NAMING PATTERN

find | read | get + By + <Field> + <Condition> + OrderBy + <Field> + <Direction>

BEST PRACTICES

- Use derived queries for simple lookups; @Query for complex ones.
- Keep method names readable; avoid very long names.

KEY TAKEAWAY Derived query methods help you write less code and focus on business logic -- Spring Data parses the method name and generates the JPQL/SQL query at runtime.

Derived Query Methods

Spring Data JPA creates queries automatically from method names by following naming conventions.

How Derived Query Methods Work

Repository Method	Generated JPQL Query	Meaning
`List<User> findByName(String name);`	`SELECT u FROM User u WHERE u.name = :name`	Find users where name equals the given name
`List<User> findByEmail(String email);`	`SELECT u FROM User u WHERE u.email = :email`	Find users where email equals the given email
`List<User> findByAgeGreaterThan(int age);`	`SELECT u FROM User u WHERE u.age > :age`	Find users where age is greater than the given age
`List<User> findByAgeLessThanEqual(int age);`	`SELECT u FROM User u WHERE u.age <= :age`	Find users where age is less than or equal to the given age
`List<User> findByEmailContaining(String email);`	`SELECT u FROM User u WHERE u.email LIKE %:email%`	Find users where email contains the given text
`List<User> findByNameAndAge(String name, int age);`	`SELECT u FROM User u WHERE u.name = :name AND u.age = :age`	Find users where name equals the given name AND age equals the given age
`List<User> findByAgeOrderByNameDesc(int age);`	`SELECT u FROM User u WHERE u.age = :age ORDER BY u.name DESC`	Find users where age equals the given age and order by name descending

How It Works

1. Define Method
You define a method in the repository interface.

2. Parse Method Name
Spring Data parses the method name based on keywords.

3. Create Query
It generates the corresponding JPQL query automatically.

4. Execute Query
The query is executed and results are returned as objects.

CUSTOM QUERIES IN SPRING DATA JPA

When derived query methods are not enough, define custom queries using @Query with JPQL, native SQL, or named queries.

@Query (JPQL) -- portable across databases

```
public interface CustomerRepository
    extends JpaRepository<CustomerEntity, Long> {

  @Query("SELECT c FROM CustomerEntity c " +
         "WHERE c.status = :status")
  Page<CustomerEntity> searchByStatus(
      @Param("status") CustomerStatus status,
      Pageable pageable);
}
```

Native SQL -- PostgreSQL-specific features

```
@Query(value = "SELECT * FROM customer c " +
    "WHERE c.status = :status " +
    "AND c.created_at >= :fromDate", nativeQuery = true)
List<CustomerEntity> findActiveSince(
    @Param("status") String status,
    @Param("fromDate") Instant fromDate);
```

USE CASES

- Complex joins
- Aggregations
- Group by / having
- PostgreSQL functions
- DTO projections

KEY POINTS

- Prefer JPQL for portability across databases.
- Use native queries only for PostgreSQL-specific features or performance reasons.
- Always use named parameters (@Param) to avoid SQL injection.
- Use projections/DTOs to fetch only what's needed.

COMMON PITFALLS

- Missing @Param for named parameters.
- Native queries tied to a specific database.
- Returning large result sets without pagination.

KEY TAKEAWAY Use @Query to write JPQL or native SQL when derived queries aren't enough. Always use parameters to avoid SQL injection; test custom queries thoroughly.

Custom Queries

Custom queries allow you to define your own JPQL or native SQL queries when derived query methods are not enough.

Calls

Executes

1. Repository Interface

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

   @Query("SELECT e FROM Employee e WHERE e.department = :dept "
          + "AND e.salary > :salary")
   List<Employee> findByDepartmentAndSalary(@Param("dept") String dept,
                                            @Param("salary") Double salary);

   @Query(value = "SELECT * FROM employees e "
          + "WHERE e.hire_date >= :date ORDER BY e.salary DESC",
          nativeQuery = true)
   List<Employee> findHiredAfterOrderBySalary(@Param("date") LocalDate date);
}
```

2. JPQL Custom Query

```
@Query("SELECT e FROM Employee e "
       WHERE e.department = :dept "
       AND e.salary > :salary")
```

- Database independent
- Uses entity names and attributes
- Validated at startup

3. Native SQL Custom Query

```
@Query(value = "SELECT * FROM employees e "
               WHERE e.hire_date >= :date "
               ORDER BY e.salary DESC",
               nativeQuery = true)
```

- Uses database specific SQL
- Supports complex queries and functions
- More control over performance

4. Result

Query Execution

Results returned to application

List<Employee>

id	name	department	salary
1	Alice	IT	85000
2	Bob	IT	90000
3	Carol	HR	70000
...	...	...	...

Key Points

- Use @Query to define custom JPQL or native SQL queries.
- Use JPQL for portability and easier maintenance.
- Use Native SQL for complex or database-specific requirements.
- Custom queries provide maximum flexibility when derived queries fall short.

PAGINATION IN SPRING DATA JPA

Pagination retrieves a bounded subset of data -- Pageable defines the request, Page<T> returns data plus metadata.

CustomerRepository + Controller (Lab 39 pattern)

```
Page<CustomerEntity> findByStatus(
    CustomerStatus status, Pageable pageable);

// In the controller: bound and cap the page size
int safeSize = Math.min(Math.max(size, 1), 100);
Pageable page = PageRequest.of(number, safeSize,
    Sort.by("fullName").and(Sort.by("id")));
Page<CustomerEntity> result =
    customerRepository.findByStatus(
        CustomerStatus.ACTIVE, page);
```

KEY CONCEPTS

Pageable — defines page number, size, and sorting.

Page<T> — result data plus total-count metadata.

Page number — zero-based index (0, 1, 2, ...).

OFFSET = page * size; LIMIT = size — how PostgreSQL executes it.

PAGE<T> METADATA

getContent() **getTotalElements()** **getTotalPages()** **hasNext()**

isFirst()/isLast()

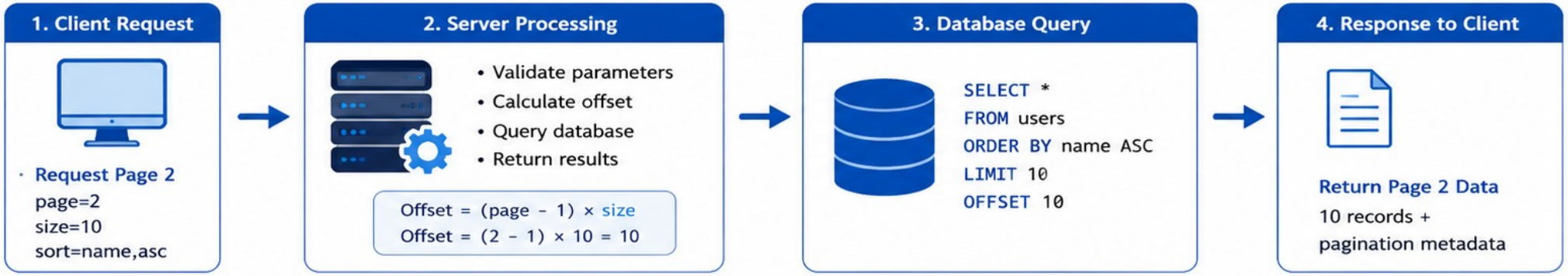
LAB 39: WHY THIS MATTERS

Lab 39 caps size at 100 (min 1) and adds an ID tie-breaker to the sort so ACTIVE customer pages stay stable and deterministic even with duplicate sort-field values.

KEY TAKEAWAY Never let clients request unbounded pages. Bound size, validate sort fields against an allow-list, and add a tie-breaker to keep paginated results deterministic.

Pagination

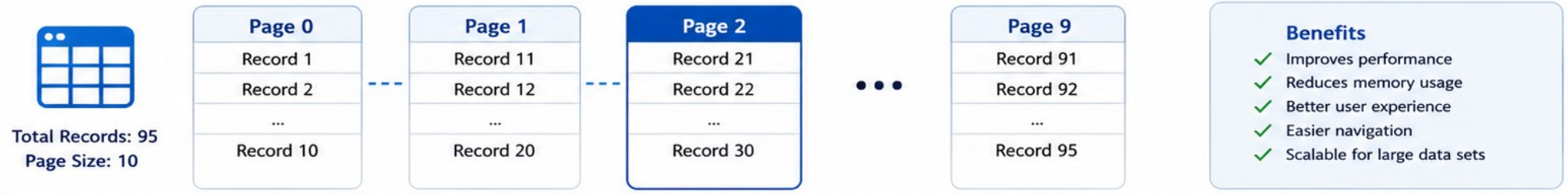
Pagination breaks large data sets into smaller chunks (pages) for efficient retrieval and display.



Example Response

```
{
  "content": [ /* 10 records for page 2 */ ],
  "page": 2,
  "size": 10,
  "totalElements": 95,
  "totalPages": 10,
  "hasNext": true,
  "hasPrevious": true
}
```

`content`	→	Data for current page
`page`	→	Current page number (0-based or 1-based)
`size`	→	Number of records per page
`totalElements`	→	Total number of records
`totalPages`	→	Total number of pages
`hasNext`	→	Is there a next page?
`hasPrevious`	→	Is there a previous page?



SORTING IN SPRING DATA JPA

Sorting orders query results by one or more fields in ascending (ASC) or descending (DESC) order, and combines cleanly with pagination.

Sort API examples

```
Sort.by("fullName").ascending();
Sort.by("fullName").descending();
Sort.by("fullName", "email").ascending();
Sort.by(Order.desc("createdAt"));

// Multi-field with mixed directions
Sort.by("status").ascending()
    .and(Sort.by("fullName").descending());

// Repository method accepting Sort
List<CustomerEntity> findByStatus(
    CustomerStatus status, Sort sort);
```

Sorting combined with paging (Lab 39)

```
Pageable pageable = PageRequest.of(0, 10,
    Sort.by("fullName").ascending()
        .and(Sort.by("id").ascending()));
Page<CustomerEntity> page =
    customerRepository.findByStatus(
        CustomerStatus.ACTIVE, pageable);
```

KEY CONCEPTS

- Sort defines field(s) and direction (ASC/DESC).
- Multiple fields can be sorted together, in order.
- Sort can be combined with pagination via PageRequest.

BEST PRACTICES

- Use entity field names, not raw column names.
- Validate/allow-list sort fields from user input.

KEY TAKEAWAY Provide default sorting to ensure consistent results, combine sorting with pagination for performance, and never trust unvalidated sort fields from client input.

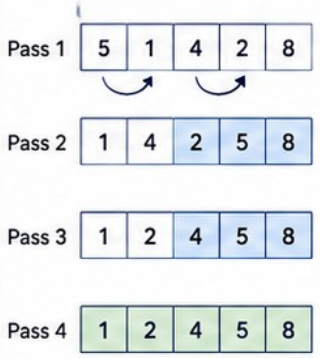
Sorting

Sorting is the process of arranging elements in a particular order (ascending or descending) to make searching, processing, and analyzing data easier.

COMMON SORTING ALGORITHMS

Bubble Sort

Repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order.

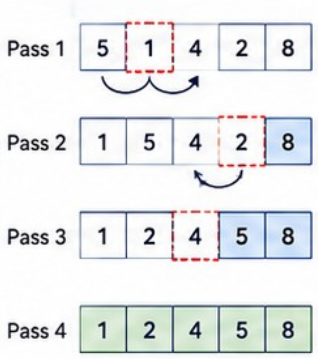


Time Complexity
Best: O(n)
Average: O(n^2)
Worst: O(n^2)

Space Complexity
O(1)

Selection Sort

Finds the minimum element from the unsorted part and places it at the beginning.

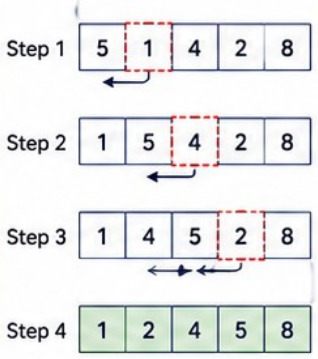


Time Complexity
Best: O(n^2)
Average: O(n^2)
Worst: O(n^2)

Space Complexity
O(1)

Insertion Sort

Builds the sorted list one item at a time by inserting each element into its correct position.

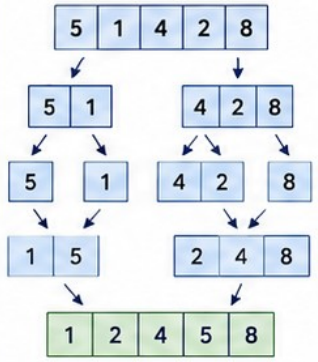


Time Complexity
Best: O(n)
Average: O(n^2)
Worst: O(n^2)

Space Complexity
O(1)

Merge Sort

Divides the list into halves, sorts each half, and then merges the sorted halves.

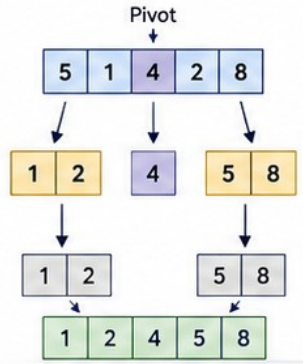


Time Complexity
Best: O(n log n)
Average: O(n log n)
Worst: O(n log n)

Space Complexity
O(n)

Quick Sort

Selects a pivot element and partitions the list into elements less than pivot and greater than pivot, then sorts them recursively.

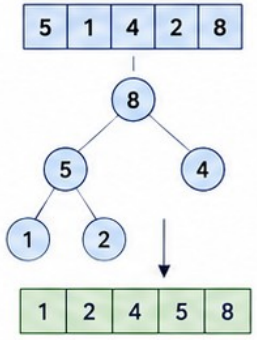


Time Complexity
Best: O(n log n)
Average: O(n log n)
Worst: O(n^2)

Space Complexity
O(log n)

Heap Sort

Builds a heap from the list, then repeatedly extracts the maximum element and rebuilds the heap.



Time Complexity
Best: O(n log n)
Average: O(n log n)
Worst: O(n log n)

Space Complexity
O(1)

Unsorted Elements | Sorted Elements (Intermediate) | Sorted Elements (Final) | Partitioned Elements | Pivot Element

TRY IT YOURSELF — EXERCISE 4: PAGING AND LOCKING NOTES

Reinforces: the Pageable and Sort mechanics from the last two slides, plus a preview of the @Version optimistic locking used throughout Lab 39.

STEPS (IN notes/lab39-paging.md)

Step	What To Do
1 — Build the Page Request	Write PageRequest.of(0, 20, Sort.by("customerId")).
2 — Plan the Response	Plan to return totalElements plus the content slice to the UI later.
3 — Document the Lock	Write: a second writer on Amina fails if the version is stale -- the user retries.
4 — Note the Correlation ID	Log lab-request-001 on every lock failure for support traceability.

Paging + optimistic-lock notes

```
Pageable page = PageRequest.of(0, 20, Sort.by("customerId"));
Page<CustomerEntity> result = customerRepository.findByStatus("ACTIVE", page);

// Concurrent update on Ravi (CUS-1002): stale @Version ->
// OptimisticLockingFailureException -> 409 + correlationId lab-request-001
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-paging-locking.md (workspace: examples/module-39-exercises).

JPQL FUNDAMENTALS

JPQL (Java Persistence Query Language) is an object-oriented query language used to query entities and their relationships -- database independent.

JPQL	SQL
Operates on Entities and Attributes	Operates on Tables and Columns
Database independent, portable	Database specific
Supports polymorphism	No polymorphism
Parsed and validated at startup	Executed directly by the database

JPQL in a repository (@Query)

```
public interface CustomerRepository
    extends JpaRepository<CustomerEntity, Long> {

  @Query("SELECT c FROM CustomerEntity c " +
         "WHERE c.status = :status " +
         "ORDER BY c.fullName ASC")
  List<CustomerEntity> findStatusOrdered(
      @Param("status") CustomerStatus status);
}
```

BASIC JPQL QUERIES

Use Case	JPQL Query	Description
Find all	SELECT c FROM CustomerEntity c	Retrieve all customers.
Find by property	SELECT c FROM CustomerEntity c WHERE c.publicId = :id	Find customer by public ID.
Multiple conditions	... WHERE c.status = :status AND c.fullName LIKE :name	Combine conditions with AND/OR.
Join	SELECT a FROM AccountEntity a JOIN a.customer c WHERE c.publicId = :id	Navigate the relationship via JOIN.

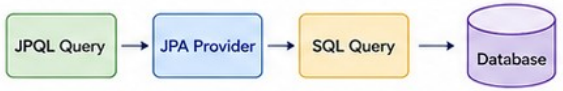
KEY TAKEAWAY JPQL operates on entities, not tables -- database independent, supports rich queries, joins, and aggregations, and is parsed and validated by the JPA provider at startup.

JPQL Fundamentals

JPQL (Java Persistence Query Language) is the object-oriented query language of JPA.
It operates on entities and their relationships, not on database tables.

1. What is JPQL?

- High-level query language for JPA.
- Works with entities and their attributes.
- Database independent.
- Parsed at runtime and converted to SQL by JPA provider.



2. Basic Syntax

```
SELECT select_clause
FROM entity_name [alias]
WHERE condition
[GROUP BY group_field]
[HAVING condition]
[ORDER BY order_field [ASC | DESC]]
```

3. Clauses

SELECT	Specifies the select list.
FROM	Specifies the entity to query.
WHERE	Filters the results.
GROUP BY	Groups the results.
HAVING	Filters grouped results.
ORDER BY	Sorts the results.

4. Examples

Select All

```
SELECT e FROM Employee e
```

Select with Where Clause

```
SELECT e FROM Employee e WHERE e.department = 'IT'
```

Select Specific Fields

```
SELECT e.name, e.salary FROM Employee e
```

Order By

```
SELECT e FROM Employee e ORDER BY e.salary DESC
```

Group By and Having

```
SELECT e.department, COUNT(e)
FROM Employee e
GROUP BY e.department
HAVING COUNT(e) > 5
```

Join

```
SELECT e.name, d.name
FROM Employee e JOIN e.department d
WHERE d.location = 'New York'
```

Named Parameter

```
SELECT e FROM Employee e WHERE e.salary > :minSalary
```

Positional Parameter

```
SELECT e FROM Employee e WHERE e.salary > ?1
```

5. Key Features

Entity Based	Queries are written using entity names and properties, not table and column names.
Relationship Navigation	Easily navigate associations. `e.department.name`
Database Independence	Portable across different databases.
Type Safety	Checked at runtime (not at compile time).
Performance	Optimized and translated to SQL by JPA provider.

6. Best Practices

- Use parameters instead of literals.
- Avoid SELECT * (select only required fields).
- Use JOIN FETCH to solve N+1 select problem.
- Paginate large result sets.
- Keep queries readable and maintainable.

7. JPQL vs SQL

JPQL	SQL
Works with entities	Works with tables
Database independent	Database specific
Uses entity attributes	Uses table columns
Supports relationship navigation	Requires joins
Parsed by JPA provider	Executed directly by DB

Tip: Always test your JPQL queries and check the generated SQL to understand performance.

LAZY VS EAGER LOADING

Fetch strategy in JPA defines when related entities are loaded from the database -- LAZY (on demand) or EAGER (immediately).

Aspect	LAZY	EAGER	Risk
When Fetched	On first access	Along with parent entity	-
Default For	@OneToMany, @ManyToMany (collections)	@ManyToOne, @OneToOne (singular)	-
Query Behavior	May trigger an additional query	Uses JOIN (single query)	-
Performance	Better when related data is not needed	Can be slower (fetches more)	-
Risk	LazyInitializationException if accessed outside a session/transaction	n+1 query problem if used carelessly	See Common Pitfalls

N+1 problem: EAGER or improper LAZY access

```
List<CustomerEntity> customers = repo.findAll();  // 1 query
for (CustomerEntity c : customers) {
  c.getAccounts().size();  // +1 query PER customer
}
// Fix: @Query("SELECT c FROM CustomerEntity c JOIN FETCH c.accounts")
```

BEST PRACTICES

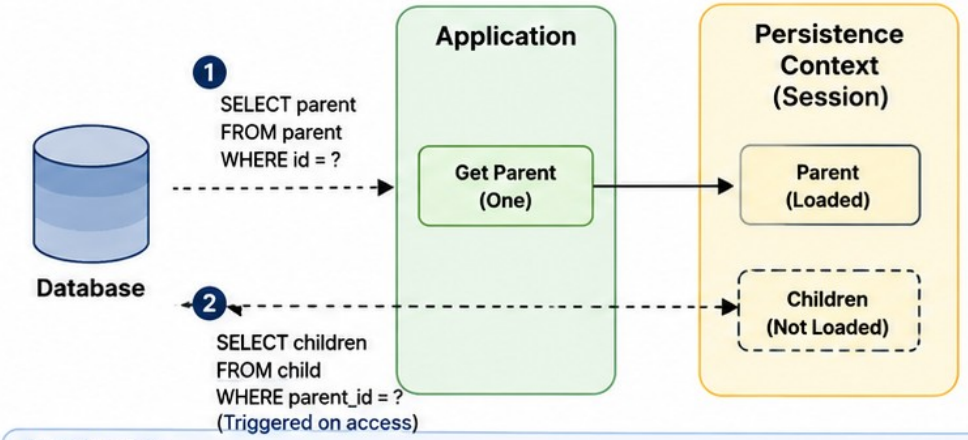
- Use LAZY as the default for associations.
- Use EAGER only when related data is always required.
- Use JOIN FETCH in JPQL to fetch related data when needed, in one query.
- Avoid accessing LAZY associations outside a transaction.

KEY TAKEAWAY Lab 39 uses LAZY for @ManyToOne on AccountEntity.customer deliberately -- overriding JPA's EAGER default -- to avoid loading the parent customer on every account query.

Lazy vs Eager Loading

LAZY LOADING

Load related data only when it is accessed



How it works

- Parent entity is loaded first.
- Child entities are not loaded initially.
- When the association is accessed, a separate query is executed.
- Better performance when related data is not always needed.

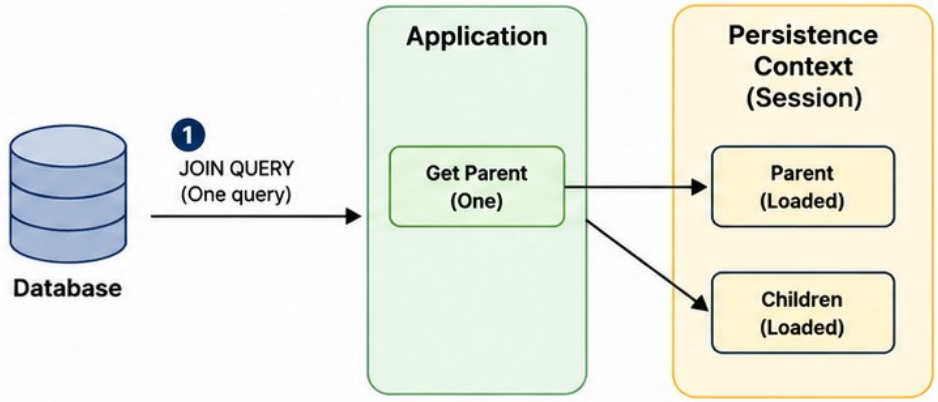
Use When

Related data is large or not always required.

VS

EAGER LOADING

Load related data immediately with the parent



How it works

- Parent and related entities are loaded in the same query (using JOIN).
- All required data is available immediately.
- Can impact performance if related data is large and not always used.

Use When

Related data is always needed and relatively small.

Key Differences	Lazy Loading	Eager Loading
When data is loaded	On first access	Immediately with parent
Number of queries	Multiple (potentially)	One (with JOIN)
Performance	Better when related data is not always used	Better when related data is always needed and small
Use case	Large associations, optional usage	Small associations, always required

CONNECTING SPRING BOOT TO POSTGRESQL

Spring Boot makes it easy to connect and interact with PostgreSQL using Spring Data JPA and Hibernate -- minimal setup, environment-based credentials.

1. Maven dependencies

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
<!-- + spring-boot-starter-data-jpa, flyway-core,
     flyway-database-postgresql (Lab 39) -->
```

2. application.yml (Lab 39 pattern -- env-based)

```
spring:
  datasource:
    url: ${CRM_DB_URL:jdbc:postgresql://
      localhost:5432/crm}
    username: ${CRM_DB_USERNAME:crm_app}
    password: ${CRM_DB_PASSWORD}
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate
  flyway:
    enabled: true
```

3. PREREQUISITES

JDK 21 | Spring Boot 3.x | PostgreSQL | Maven 3.9+

4. WHY EACH SETTING MATTERS

ddl-auto: validate — Hibernate checks the schema matches, never mutates it.

open-in-view: false — forces lazy access inside the transaction.

${CRM_DB_PASSWORD} — credentials from environment, never hardcoded.

COMMON ISSUES & SOLUTIONS

Symptom	Fix
Connection refused	Check PostgreSQL server status/port (5432).
Authentication failed	Verify username/password in env vars.
Driver not found	Add the postgresql runtime dependency.

KEY TAKEAWAY Externalize datasource credentials; use connection pooling (HikariCP is Spring Boot's default); never hardcode credentials in source; use ddl-auto: validate, not update/create.

Connecting Spring Boot to PostgreSQL

Spring Data JPA (or JDBC)

1. Spring Boot Starts
2. DataSource is configured
3. Connects to PostgreSQL
4. Uses Spring Data JPA / JDBC

Spring Boot Application

JDBC / Hibernate (DataSource)

PostgreSQL Database

application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=myuser
spring.datasource.password=mypassword
spring.jpa.hibernate.ddl-auto=update
```

TRY IT YOURSELF — EXERCISE 5: FLYWAY PLAN

Reinforces: the flyway.enabled and ddl-auto: validate settings you just saw, applied to planning the real V1__crm_schema.sql migration.

STEPS (IN notes/lab39-flyway.md)

Step	What To Do
1 — Name the Version File	V1__crm_schema.sql, placed under the conventional db/migration folder.
2 — Plan the Content	Include the customer + account DDL from the Lab 37 design.
3 — State Why Flyway	One sentence: schema changes are versioned and repeatable across every machine.
4 — Flag the Anti-Pattern	Avoid relying on ddl-auto=create-drop for shared environments.

V1__crm_schema.sql (planned)

```
-- db/migration/V1__crm_schema.sql
CREATE TABLE customer ( ... );   -- from Lab 37 design
CREATE TABLE account ( ... );    -- from Lab 37 design

-- Anti-pattern: spring.jpa.hibernate.ddl-auto: create-drop on a shared DB
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-flyway-plan.md (workspace: examples/module-39-exercises).

DATA PERSISTENCE WORKFLOW

Data persistence in Spring Boot with Spring Data JPA and PostgreSQL follows a well-defined workflow from the application layer down to the database and back.

1 **1. Application**
Controller/Service calls repository.

2 **2. Repository**
save/findById/delete etc.

3 **3. EntityManager**
Manages entity state.

4 **4. Hibernate**
Translates to SQL.

5 **5. PostgreSQL**
Executes SQL, persists data.

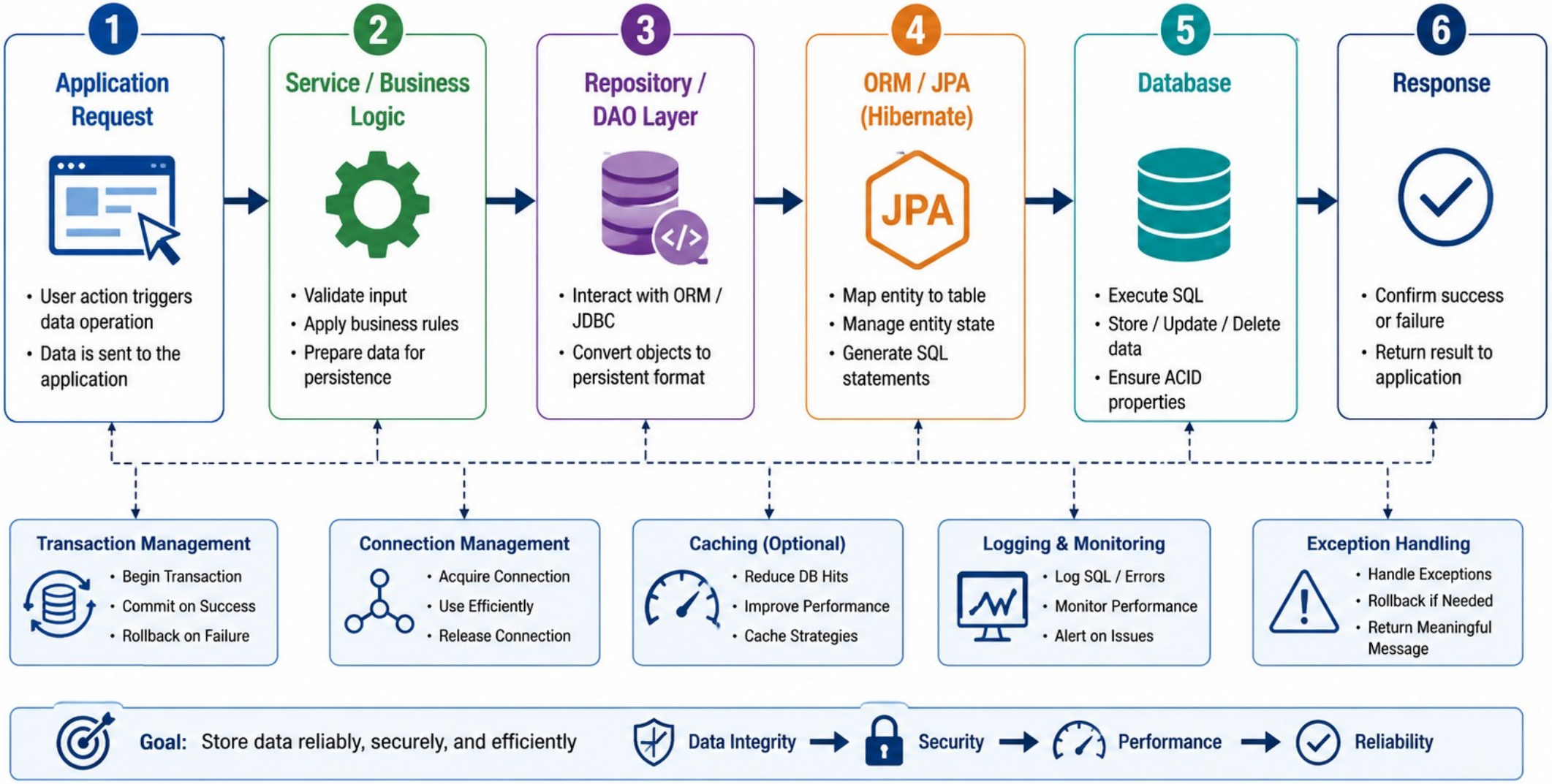
Operation	Workflow	Description
Create	Controller -> Repository.save() -> EntityManager.persist() -> Hibernate INSERT -> PostgreSQL	New entity inserted into the database.
Read	Controller -> Repository.findById() -> EntityManager.find() -> Hibernate SELECT -> PostgreSQL	Data queried and mapped to entity.
Update	Controller -> Repository.save() -> EntityManager.merge() -> Hibernate UPDATE -> PostgreSQL	Existing entity updated in the database.
Delete	Controller -> Repository.deleteById() -> EntityManager.remove() -> Hibernate DELETE -> PostgreSQL	Entity deleted from the database.

FLOW SUMMARY

Application request -> Repository -> EntityManager -> Hibernate -> PostgreSQL -> results returned back to the application.

KEY TAKEAWAY All write operations occur inside a transaction; use @Transactional on service-layer methods; enable spring.jpa.show-sql=true for debugging.

Data Persistence Workflow



TRANSACTION INTEGRATION

Spring integrates Spring Data JPA transactions with Hibernate and PostgreSQL to ensure data consistency, integrity, and reliability.

ACID PROPERTIES

Property	Meaning
Atomicity	All operations succeed, or none do.
Consistency	Database remains in a valid state.
Isolation	Concurrent transactions don't interfere.
Durability	Committed data persists after failure.

@Transactional (Lab 39 pattern)

```
@Service
public class CustomerService {
  @Transactional
  public CustomerResponse create(
      CreateCustomerRequest req) {
    String email = normalize(req.email());
    if (repo.existsByNormalizedEmail(email)) {
      throw new DuplicateCustomerException();
    }
    var saved = repo.save(mapper.toEntity(req));
    return mapper.toResponse(saved);
  }
}
```

PROPAGATION (COMMON)

Type	Behavior
REQUIRED	Join existing or create new (default).
REQUIRES_NEW	Always creates a new transaction.
MANDATORY	Joins existing; throws if none exists.
NESTED	Nested transaction with savepoints.

ROLLBACK RULES

- By default, Spring rolls back on RuntimeException and Error.
- Checked exceptions do NOT trigger rollback unless specified via rollbackFor.
- Define transactions at the service layer, not in repositories.

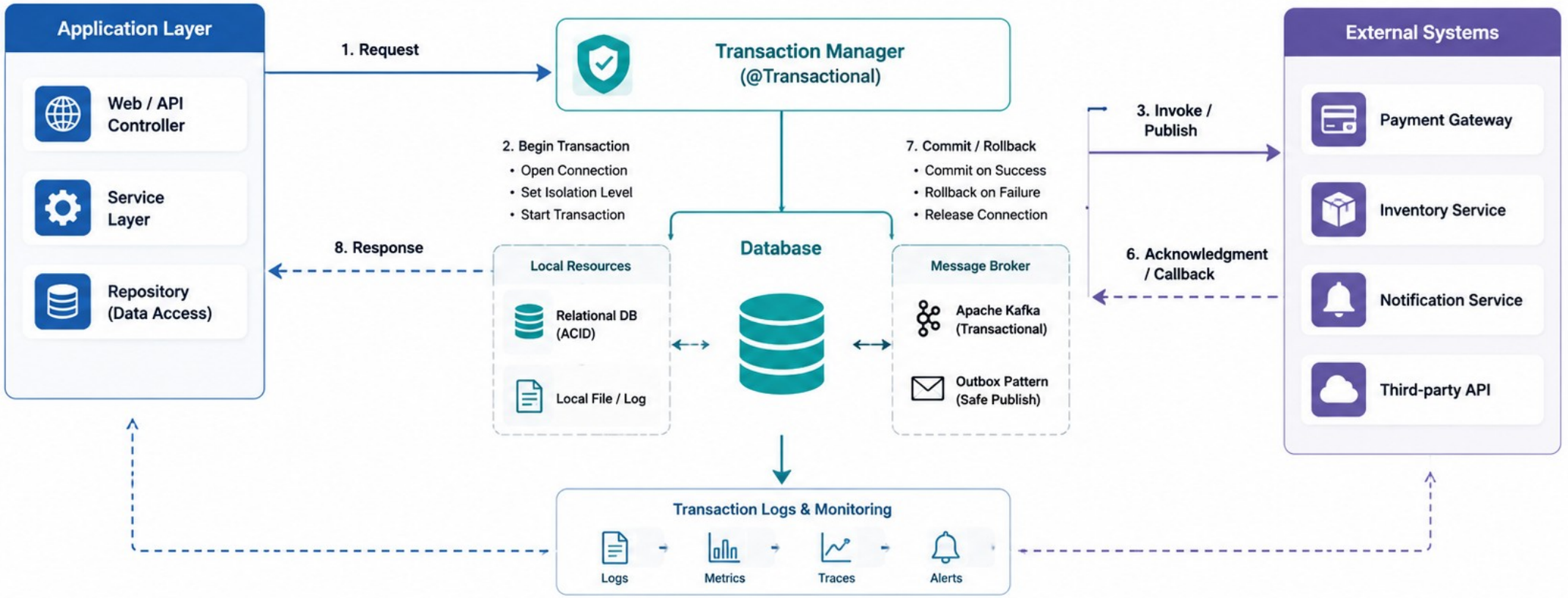
ISOLATION LEVELS (COMMON)

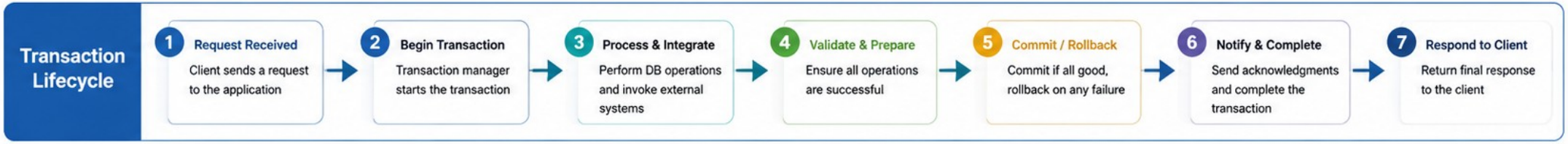
READ_COMMITTED (PostgreSQL default) **REPEATABLE_READ**

SERIALIZABLE

KEY TAKEAWAY @Transactional at the service layer ensures atomicity and consistency; keep transactions short and focused; always test commit, rollback, and exception scenarios.

Transaction Integration





PERFORMANCE OPTIMIZATION IN JPA

Optimize your JPA applications against PostgreSQL for scalability, speed, and efficient resource usage.

1. Fetch Strategies

Use LAZY by default; use JOIN FETCH in queries when related data is genuinely needed.

2. Batch Processing

Batch similar inserts/updates; enable hibernate.jdbc.batch_size.

3. Projections

Fetch only the fields you need via DTO or interface-based projections.

4. Pagination

Always paginate large result sets; avoid loading huge collections.

5. Second-Level Cache

Cache frequently-read, rarely-changed data with a provider like Caffeine.

6. Query Optimization

Write efficient JPQL/SQL; avoid functions on indexed WHERE columns.

7. Indexing

Create PostgreSQL indexes for columns used in WHERE, JOIN, ORDER BY.

8. Flush Mode

Use FlushModeType.COMMIT for read-only transactions to avoid unnecessary flushes.

N+1 problem: fix with JOIN FETCH

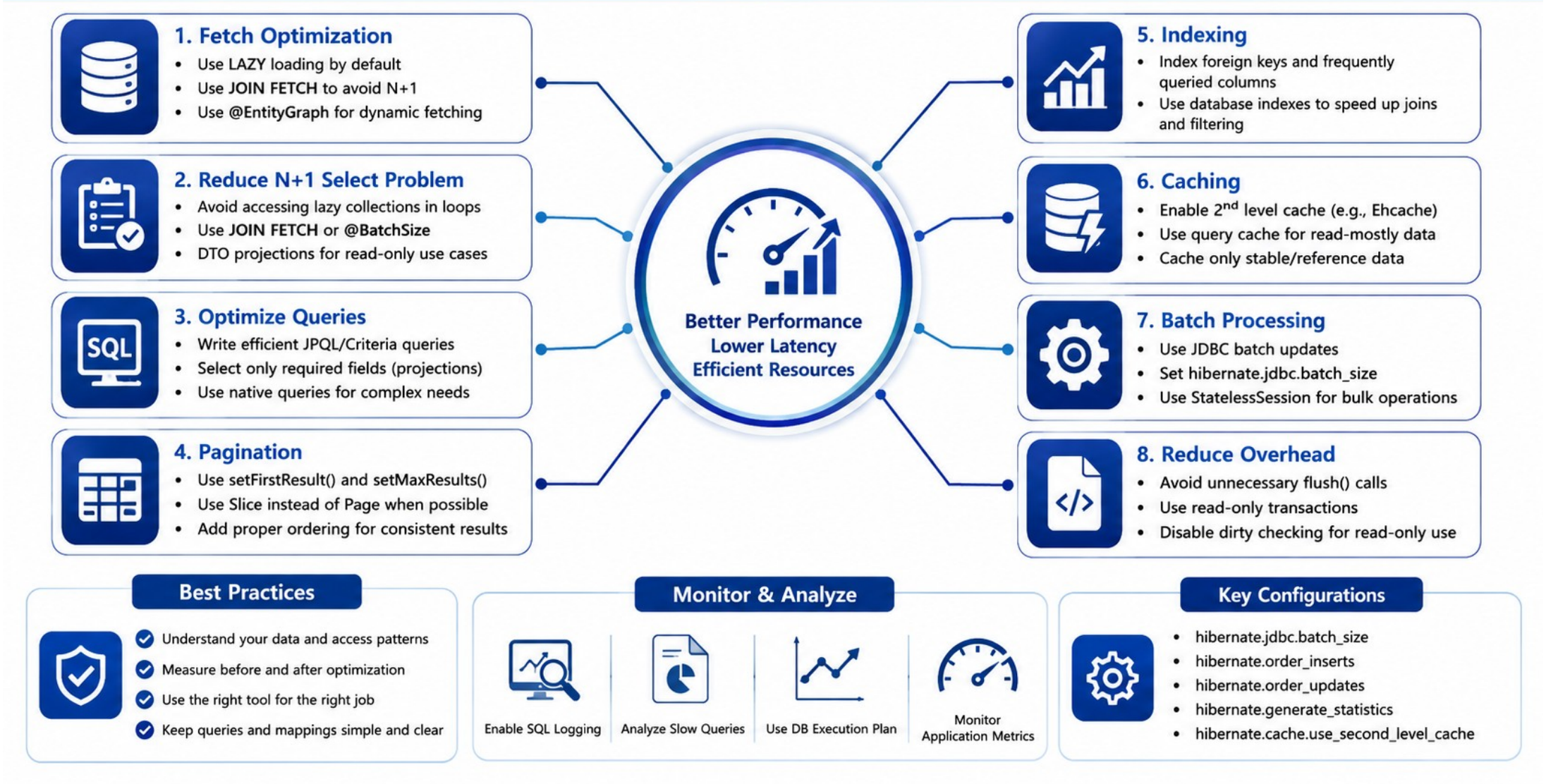
```
// Problem: 1 + N queries
customerRepo.findAll().forEach(c -> c.getAccounts().size());

// Solution: 1 query total
@Query("SELECT c FROM CustomerEntity c LEFT JOIN FETCH c.accounts")
List<CustomerEntity> findAllWithAccounts();
```

MEASURING & MONITORING

- Enable spring.jpa.show-sql=true and hibernate.format_sql for analysis.
- Use Hibernate statistics (hibernate.generate_statistics=true).
- Monitor query count, execution time, and cache hit ratio.

KEY TAKEAWAY Default to LAZY loading, use JOIN FETCH only when necessary, paginate all large queries, use DTO projections for read-heavy endpoints, and always measure before optimizing.

Performance Optimization in JPA



COMMON ORM PITFALLS (1 OF 2)

Avoid these frequent mistakes when working with JPA/Hibernate against PostgreSQL to improve performance, correctness, and maintainability.

1. N+1 SELECT PROBLEM

```
// Problem: 1 + N queries
List<CustomerEntity> cs = repo.findAll();
for (var c : cs) c.getAccounts().size();  // +1 each
```

Better approach — use JOIN FETCH or @EntityGraph to fetch related data in one query.

2. EAGER FETCHING BY DEFAULT

```
// Problem: forces loading even when not needed
@ManyToOne(fetch = FetchType.EAGER)
private CustomerEntity customer;
```

Better approach — use LAZY (as Lab 39's AccountEntity does) and fetch only when needed.

3. MISSING / WRONG INDEXES

```
-- Problem: full table scan if no index on status
SELECT * FROM customer WHERE status = 'ACTIVE';
```

Better approach — CREATE INDEX ix_customer_status ON customer(status);

4. IGNORING TRANSACTIONS

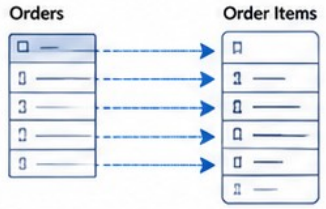
```
// Problem: without transaction boundaries,
// data can be left inconsistent on partial failure
customerRepo.save(c); accountRepo.save(a);  // no @Transactional
```

Better approach — wrap the service method in @Transactional to ensure atomicity.

KEY TAKEAWAY N+1 queries, EAGER-by-default, missing indexes, and skipped transactions are the four most common correctness and performance killers in real JPA applications.

Common ORM Pitfalls

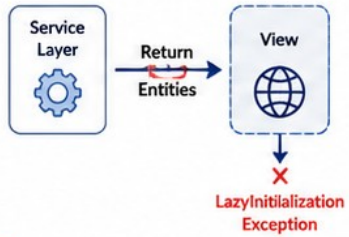
1 N+1 Query Problem



Problem: ORM issues one query for parent entities, then N additional queries for child entities.

Tip: Use fetch join, entity graphs, or batch fetching.

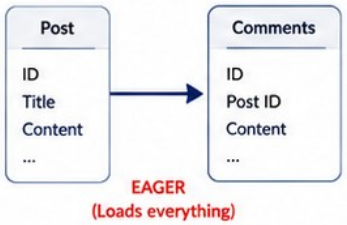
2 Lazy Loading Pitfalls



Problem: Accessing lazy-loaded associations outside the session causes exceptions.

Tip: Initillaize needed data within transaction or use DTOs.

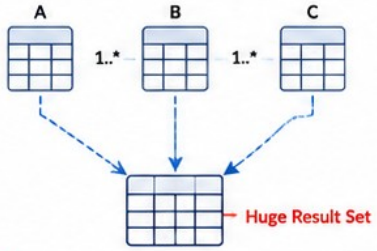
3 Over-Fetching (Eager Loading)



Problem: Eager loading fetches more data than needed, impacting performance.

Tip: Prefer lazy loading and fetch only what is necessary.

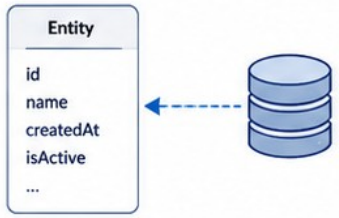
4 Cartesian Product



Problem: Joining multiple collections can produce huge result sets (cartesian explosion).

Tip: Avoid joining multiple collections; split queries or use batch fetching.

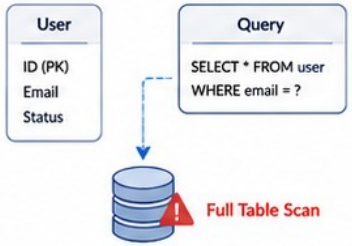
5 Leaky Abstractions



Problem: Application code becomes too dependent on persistence details.

Tip: Keep domain model clean; use DTOs and repositories.

6 Missing Indexes



Problem: ORM hides queries; missing indexes lead to full table scans.

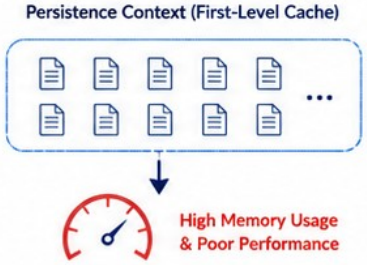
Tip: Analyze queries and add proper indexes.

7 Cascade Misuse

Problem: Wrong cascade settings can delete or update more data than intended.

Tip: Use cascade types carefully; prefer explicit operations.

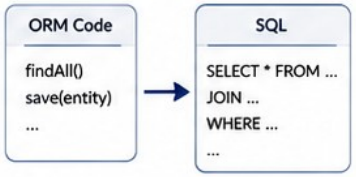
8 Large Persistence Context



Problem: Loading too many entities in one session causes memory bloat.

Tip: Use pagination, clear context, or separate transactions.

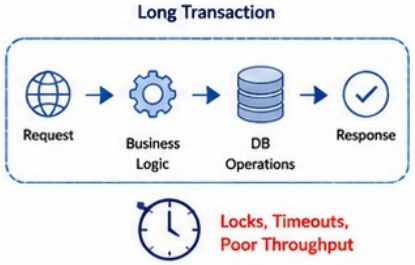
9 Ignoring Generated SQL



Problem: Not checking generated SQL can hide inefficient or incorrect queries.

Tip: Enable SQL logging and review queries regularly.

10 Improper Transaction Boundaries



Problem: Long transactions hold locks and degrade system performance.

Tip: Keep transactions short and focused.

COMMON ORM PITFALLS (2 OF 2)

Four more frequent JPA/Hibernate mistakes -- unnecessary updates, improper cascading, large object graphs, and LazyInitializationException.

5. UNNECESSARY UPDATES

```
// Problem: Hibernate issues UPDATE even when
// the value hasn't actually changed
customer.setFullName(customer.getFullName());
```

Better approach — only modify when the value actually changes, or use @DynamicUpdate.

6. IMPROPER CASCADING

```
// Problem: CascadeType.ALL can cause
// unintended deletes
@OneToMany(mappedBy="customer", cascade=CascadeType.ALL)
```

Better approach — use only the specific cascade types actually required (e.g., PERSIST, MERGE).

7. LARGE OBJECT GRAPHS

```
// Problem: fetching huge graphs loads too
// much data into memory
List<CustomerEntity> all = repo.findAll();  // every account too
```

Better approach — fetch selectively via projections or pagination.

8. LAZYINITIALIZATIONEXCEPTION

```
// Problem: accessing lazy data after the
// session/transaction has closed
customer.getAccounts().size();  // throws outside TX
```

Better approach — fetch within the transaction, via JOIN FETCH or a DTO mapping.

KEY TAKEAWAY Always measure before optimizing -- use logs, metrics, and load tests to identify real bottlenecks. Higher DB load, slower responses, and hard-to-maintain code are the cost of ignoring these pitfalls.

JPA BEST PRACTICES (1 OF 2)

Follow these best practices to build efficient, maintainable, and scalable JPA applications against PostgreSQL.

1. Prefer LAZY Loading

EAGER loading fetches unnecessary data and hurts performance. Use LAZY for @OneToMany, @ManyToMany, and @OneToOne associations; load related data only when needed.

2. Use DTO Projections

Fetching entire entities when you need only a few fields wastes resources. Use DTO or interface-based projections for read-only use cases.

3. Write Efficient Queries

Poor queries lead to N+1 problems and slow performance. Use JOIN FETCH, @EntityGraph, or a tuned batch size to minimize N+1 queries.

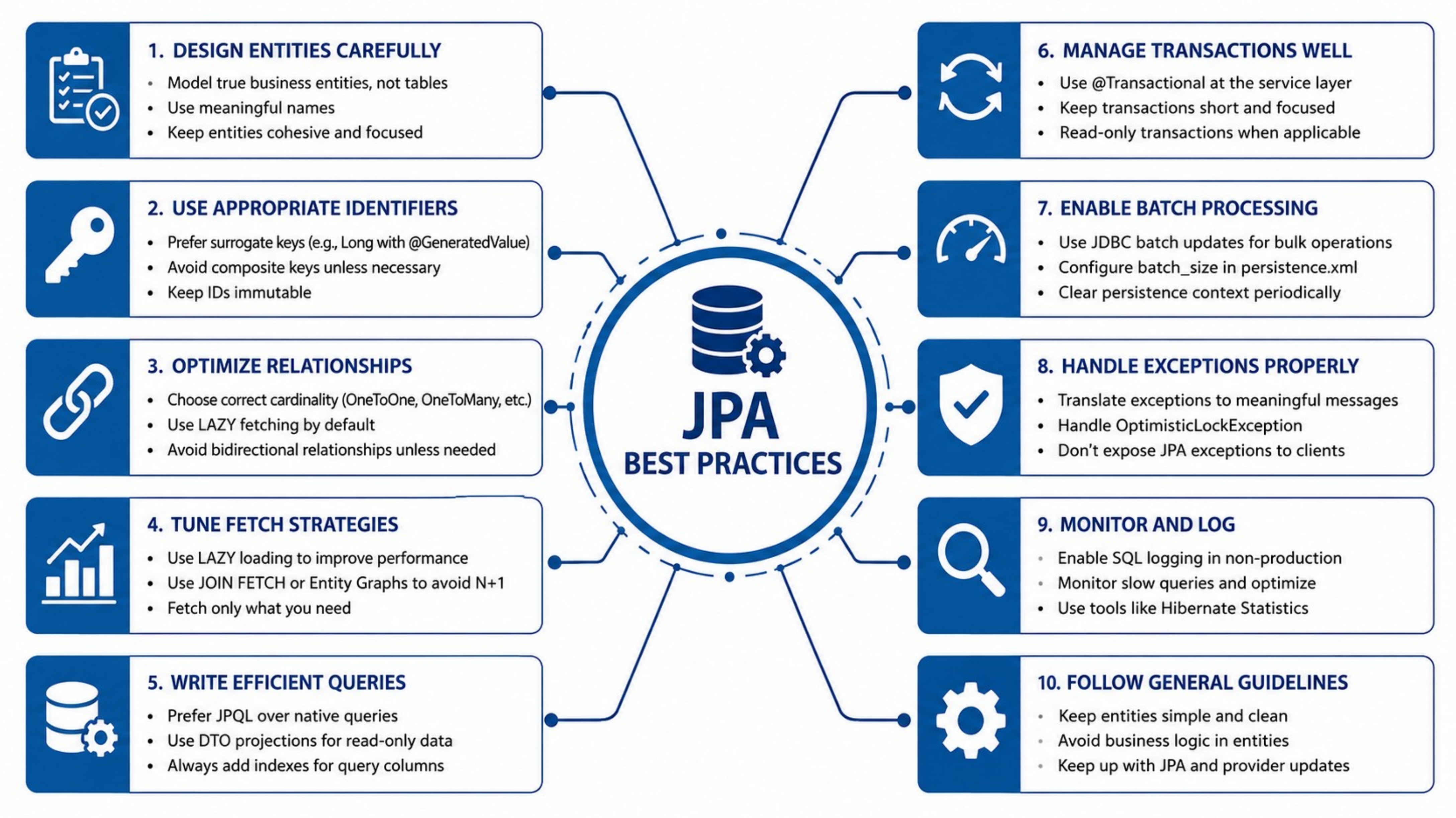
4. Use Pagination & Sorting

Loading large result sets into memory causes performance issues. Use Pageable for pagination and sorting on every large query.

5. Enable & Use Caching

Repeated queries on the same data hit the database again and again. Use second-level cache for frequently-read data; choose a provider like Caffeine.

KEY TAKEAWAY Performance, consistency, maintainability, security, and scalability are the five core principles every JPA best practice below supports.



JPA BEST PRACTICES (2 OF 2)

Five more best practices for correct relationships, safe transactions, and production-ready PostgreSQL-backed applications.

6. Maintain Proper Relationships

Incorrect mappings lead to data issues and performance penalties. Define correct cardinality; use proper cascading and orphanRemoval.

7. Use Transactions Properly

Without transactions, data can become inconsistent on failure. Use @Transactional at the service layer with the right propagation.

8. Avoid Open Session in View

OSIV keeps the session open through view rendering, masking real issues. Disable it (spring.jpa.open-in-view=false) and fetch what's needed in the service layer.

9. Validate Input Data

Invalid data causes persistence errors and inconsistent states. Use Bean Validation (Jakarta Validation) on entities and DTOs; fail fast with meaningful errors.

10. Monitor and Tune Regularly

Applications change over time -- what's fast today may be slow tomorrow. Monitor SQL, analyze slow queries, and tune fetch/batch/index settings continuously.

KEY TAKEAWAY Good JPA design is about fetching the right data, at the right time, with the right consistency and minimal cost. Measure -> analyze -> optimize -> repeat.

TRY IT YOURSELF — EXERCISE 6: LAB 39 READINESS (CAPSTONE)

Capstone pre-lab check -- confirm your Exercises 1-5 plans are ready before Lab 39's timed starter path begins.

STEPS (IN notes/lab39-readiness.md)

Step	What To Do
1 — Confirm the Stack	JDK 21 + Maven + PostgreSQL + Flyway + Spring Data JPA (started for real in the lab).
2 — Preview the Evidence	Plan an IT test or smoke check showing Amina (CUS-1001) loaded via the repository.
3 — Confirm Secrets Hygiene	Use .env / environment variables for the datasource password -- never commit it.
4 — Mark Pass/Fail	Pass if the Exercise 3 JPA TODOs are filled and the Exercise 5 Lab 37 DDL plan exists on paper.

Pass mark rule

```
Pass if:
  1) Exercise 4's entity/repository/config TODOs are filled (not just copied), AND
  2) Exercise 2's V1__crm_schema.sql plan exists on paper.
Otherwise: return to the incomplete exercise before Lab 39 begins.
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 6 before continuing: exercises/exercise-06-lab39-readiness.md (workspace: examples/module-39-exercises). This is the final gate before Lab 39 begins.

LAB OVERVIEW -- SPRING DATA JPA WITH POSTGRESQL

Lab 39: Spring Data JPA with PostgreSQL -- Flyway, Entities, Repositories, Paging, Optimistic Lock.

BUSINESS SCENARIO

- Leadership freezes a persistence gate before security testing (Lab 40) and containers (Lab 41): Hibernate must validate schema (never create-drop), Open Session in View stays off, money uses BigDecimal.
- Duplicate email and optimistic-lock conflicts must return a controlled HTTP 409 without leaking PostgreSQL SQLSTATE/exception text to clients.
- You own that gate for CRM create/find/list/update with Amina (CUS-1001 ACTIVE), Ravi (CUS-1002 PROSPECT to ACTIVE), duplicate email, not-found CUS-9999, and concurrent update races.

LEARNING OBJECTIVES

- Configure Spring Data JPA for PostgreSQL with env-based credentials.
- Manage schema changes with Flyway (validate + migrations only).
- Map customer and account entities to PostgreSQL types accurately (BigDecimal, @Version).
- Create repository lookups, existence checks, and paging queries.
- Write transactional services that map entities to response DTOs.
- Handle unique and optimistic-lock conflicts as safe API errors.

WHAT YOU BUILD

- lab39-crm against real shared PostgreSQL; V1__crm_schema.sql Flyway migration; CustomerEntity/AccountEntity mapped with @Version optimistic locking; CustomerRepository/AccountRepository with paging; a @Transactional CustomerService with DTO mapping; an ApiExceptionHandler mapping duplicate-email and optimistic-lock conflicts to safe 409 responses; CustomerRepositoryIT proving it all against real PostgreSQL via mvn clean verify.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; correlation ID lab-request-001 appears on every conflict/error response; ddl-auto stays validate, never update/create.

LAB TASKS AND DELIVERABLES

lab39-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Start PostgreSQL and scaffold lab39-crm under java-bootcamp/examples.
2	Add JPA, PostgreSQL JDBC, and Flyway (flyway-database-postgresql) dependencies.
3	Configure PostgreSQL safely: env-based credentials, open-in-view: false, ddl-auto: validate.
4	Author the Flyway V1__crm_schema.sql migration (customer + account tables).
5	Map CustomerEntity with public_id, normalized email, status, and @Version.
6	Map AccountEntity with a BigDecimal balance and a LAZY @ManyToOne to customer.
7	Create focused repositories: findByPublicId, existsByNormalizedEmail, findByStatus paging.
8	Write the transactional service, expose deterministic bounded paging, and translate duplicate/optimistic conflicts to safe 409s.
9	Run CustomerRepositoryIT against real PostgreSQL until mvn clean verify is green.

Exception handler pattern

```
@ExceptionHandler(DataIntegrityViolationException.class)
ResponseEntity<ProblemDetail> duplicate(...) { /* 409, no SQL text */ }
@ExceptionHandler(OptimisticLockingFailureException.class)
ResponseEntity<ProblemDetail> conflict(...) { /* 409 */ }
```

EXPECTED DELIVERABLES

- Spring Boot app with PostgreSQL JPA + Flyway V1.
- CustomerEntity / AccountEntity with correct types and @Version.
- Repositories, transactional service, bounded paging controller.
- Exception handler mapping conflicts to 409.
- CustomerRepositoryIT + mvn clean verify success evidence.
- .env.example + README runbook.

RUBRIC (100 MARKS)

- Environment/Structure 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs 10

KEY TAKEAWAY Shipping ddl-auto=create-drop is an honor violation; returning entities with lazy bags to JSON "because it worked with OSIV" loses production marks; H2-only tests when PostgreSQL was available lose verification marks.

MODULE 39 SUMMARY

In this module, you learned how to build robust, scalable, and efficient data access layers in Spring Boot applications using Spring Data JPA and PostgreSQL.

KEY CONCEPTS COVERED

JPA with Spring Boot

Integrated Spring Boot with JPA to simplify data access and configuration.

PostgreSQL Integration

Connected and configured PostgreSQL as the relational database.

Entities & Relationships

Modeled entities using JPA annotations and mapped relationships effectively.

Repositories & JPQL

Used repository interfaces, derived queries, custom JPQL, projections, and paging.

Transactions

Managed transactions with @Transactional, propagation, and rollback.

Hands-on Lab

Built lab39-crm: Flyway, entities, repositories, paging, and optimistic locking.

MODULE FLOW RECAP

1. **Application**
2. **Repository (JPA)**
3. **JPA / Hibernate**
4. **PostgreSQL**
5. **Lab: lab39-crm**

KEY TAKEAWAY Spring Data JPA greatly simplifies data access with powerful abstractions. Proper entity modeling, the right fetch strategy, transactions, and best practices are what make it production-ready.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of JPA entity annotations, primary keys, relationships, and default fetch types.

1. Which annotation is used to mark a class as a JPA entity?

A. @Component
B. @Entity
C. @Table
D. @Repository

2. Which annotation is used to specify the primary key of an entity?

A. @Id
B. @PrimaryKey
C. @Column
D. @Key

3. Which of the following is NOT a valid JPA relationship?

A. @OneToOne
B. @ManyToOne
C. @OneToMany
D. @OneToManyMany

4. What is the default fetch type for @ManyToOne?

A. LAZY
B. EAGER
C. AUTO
D. IMMEDIATE

KEY TAKEAWAY @Entity marks a JPA entity; @Id defines the primary key; @OneToManyMany does not exist (the valid types are OneToOne, ManyToOne, OneToMany, ManyToMany); @ManyToOne defaults to EAGER (singular associations default EAGER, collections default LAZY).

KNOWLEDGE CHECK (2 OF 2)

Four more questions on custom JPQL queries, transaction propagation, the @Transactional annotation, and solving the N+1 problem.

5. Which annotation is used to write custom JPQL queries in a repository method?

A. @Query

B. @JPQL

C. @Select

D. @CustomQuery

6. Which of the following is NOT a valid propagation type in @Transactional?

A. REQUIRED

B. SUPPORTS

C. MANDATORY

D. SERIALIZABLE

7. Which annotation is used on a service method to make it transactional?

A. @Transactional

B. @Service

C. @Transaction

D. @EnableTransaction

8. Which of the following helps solve the N+1 select problem?

A. EAGER fetch for all relationships

B. LAZY fetch with @Query using JOIN FETCH

C. Disable second-level cache

D. Use @Transactional(readOnly = true)

KEY TAKEAWAY @Query defines custom JPQL; SERIALIZABLE is an isolation level, not a propagation type; @Transactional marks a method transactional; JOIN FETCH (not blanket EAGER) is the correct fix for N+1.

"Focus on your business logic, JPA takes care of the persistence."

MODULE 39 COMPLETE

Spring Data JPA and PostgreSQL

You can now design, map, query, and persist entities with Spring Data JPA against real PostgreSQL -- entities, relationships, repositories, JPQL, pagination, transactions, and performance best practices.